

Staatsexamen: Informatik

Alle Themen und Aufgabentypen

Paul Kube & Matthias Rips

Inhaltsverzeichnis

1	Theoretische Informatik	5
1.1	Grammatiken	5
1.1.1	Chompsky-Hierarchie	5
1.1.2	Entscheidbarkeit bei Typ 1,2,3-Sprachen	7
1.1.3	Deterministische Endliche Automaten	7
1.1.4	Potenzmengenkonstruktion	7
1.1.5	Minimierung eines DFAs	8
1.1.6	Reguläre Ausdrücke	9
1.1.7	Satz von Myhill-Nerode	9
1.2	Kontextfreie Sprachen	10
1.2.1	Chompsky-Normalform	10
1.2.2	Kellerautomaten	11
1.3	Wortproblem	11
1.3.1	CYK-Algorithmus	11
1.4	Kontextsensitive Sprachen	11
1.5	Überblick Sprachtypen	12
1.6	Berechenbarkeit und Reduktionen	12
1.6.1	Entscheidbarkeit	12
1.6.2	Reduktionsbeweise	13
2	Algorithmen und Datenstrukturen	14
2.1	Algorithmen Allgemein	14
2.1.1	Eigenschaften von Algorithmen	14
2.2	Listen und Operationen auf Listen	14
2.2.1	Sortierverfahren	14
2.2.2	Suchalgorithmen	16
2.3	Bäume	16
2.3.1	Eigenschaften in Binärbäumen	16
2.3.2	Baumtraversierungen	16
2.3.3	Binäre Suchbäume	16
2.3.4	Varianten von Binärbäumen	17
2.4	Laufzeiten	17
2.5	Hashing	18
2.5.1	Umgang mit Kollisionen	19
2.5.2	Gängige Sondierungsmethoden	19
2.6	Algorithmus von Dijkstra	19
2.7	Minimale Spannbäume	20
2.8	Algorithmische Paradigmen	20
3	Softwaresysteme	21
3.1	Projektplanung	21
3.1.1	Softwaremaße	21
3.1.2	Qualitätskriterien con Code	22
3.1.3	GANTT-Diagramme	22
3.1.4	PERT/CPM Netzwerke	22
3.2	Testen	22
3.2.1	Überdeckungsmaße	22
3.2.2	Kontrollflussgraphen	23

3.3	Prozessmodelle	23
3.3.1	Wasserfallmodell	23
3.3.2	V-Modell	24
3.3.3	Evolutionäre Methoden	24
3.3.4	Spiralmodell	24
3.3.5	Agiles Manifest	24
3.3.6	Methoden und Praktiken	25
3.4	SCRUM	25
3.4.1	Artefakte	25
3.4.2	Rollen	26
3.4.3	Ablauf	26
3.5	Designpatterns	26
3.5.1	MVC	26
3.5.2	Observer	27
3.5.3	Composite	27
3.5.4	Factory	28
3.5.5	Adapter	28
3.5.6	Singleton	28
3.6	UML-Diagramme	29
3.6.1	Use-Case-Diagramme	30
3.6.2	Klassendiagramme	31
3.6.3	Sequenzdiagramme	33
3.6.4	Zustandsdiagramme	34
3.6.5	Aktivitätendiagramm	35
3.7	Allgemeine Reste	35
3.7.1	User Stories	35
3.7.2	Refactoring	35
4	Datenbanken	36
4.1	Relationales Modell	36
4.1.1	Begriffe	36
4.1.2	Vermischtes	36
4.2	DDL	37
4.3	Realtionale Algebra	37
4.3.1	Grundoperationen	37
4.3.2	JOINS	37
4.3.3	Divisionen	38
4.3.4	SQL	38
4.3.5	Tupel-Kalkül	41
4.3.6	Bereichskalkül	42
4.4	E-R-Modell	42
4.4.1	Elemente des E-R-Modells	42
4.4.2	Funktionalität von Reactions	43
4.4.3	Schwache Entitäten	43
4.4.4	Vererbung in E-R	43
4.4.5	Übersetzung E-R zum relationalen Modell	44
4.5	Normalformen	44
4.5.1	Funktionale Abhängigkeit	44
4.5.2	Attributhülle	45

4.6	Anfragenoptimierung	47
4.7	Transaktionen	49
4.7.1	Eigenschaften	49
4.7.2	Probleme der Synchronisation	49
4.7.3	Schedules	50
4.7.4	2-Phasen-Sperrprotokoll	51

1 Theoretische Informatik

Das müssen wir alles thematisieren:

- Schreibweisen bei Sprachen und Grammatiken
- Myhill-Nerode
- Chompsky-Hierarchie (frei, kontextfrei, regulär)
- Regex, CYK-Algorithmus
- Entscheidbarkeit (v.a. was ist Gödelnummer, Gödelisierung einer mit w codierten Turingmaschine)
- P/NP Begriffsfeld
- Turingmaschinen
- Reduktionen
- Arten von Automaten (denk auch an Kellerautomat)
- Minimaler Automat (Minimierungstabelle), Potenzmengenkonstruktion, DFA Konstruktion

1.1 Grammatiken

Definition: Notationen

- Eine Sprache L über einem Alphabet Σ , ist eine Teilmenge von Σ^* .
- L^* bezeichnet den **Kleenschen Abschluss**:

$$L^* := \bigcup_{i \geq 0} L^i$$

- L^+ ist der Kleensche Abschluss nur ohne dem leeren Wort.

Definition: Erzeugte Grammatik

Die von einer Grammatik $G = (V, \Sigma, P, S)$ ist die erzeugte Sprache $L(G)$ mit:

$$L(G) := \{w \in \Sigma^* : S \Rightarrow_G^* w\}$$

1.1.1 Chompsky-Hierarchie

Die Grammatiken wurden von Chompsky in verschiedene Klassen unterteilt:

Definition: Chomsky-Hierarchie

Sei $G = (V, \Sigma, P, S)$ eine Grammatik

- Jede Grammatik ist automatisch vom Typ 0.
- Eine Grammatik ist **kontextsensitiv**, wenn gilt: $\forall(l \rightarrow r) \in P: |l| \leq |r|$.
- Eine Grammatik ist **kontextfrei**, wenn gilt: $\forall(l \rightarrow r) \in P: l = A \in V$.
- Eine Grammatik ist **regulär**, wenn sie kontextfrei ist und wenn gilt: $\forall(A \rightarrow r) \in P: r = a \vee r = aA', A' \in V$.

Für Typ-2-Grammatiken gibt es immer eine Rechts- und Linksableitung. Das heißt, es wird entweder erst das linkeste oder rechteste nicht-Terminal abgeleitet.

Für alle diese Grammatiken gibt es die ϵ -Regel, die es erlaubt, dass das Startsymbol einmal in das leere Wort abgeleitet werden darf. Die restlichen Eigenschaften gehen dadurch nicht verloren.

Um zu zeigen, dass eine Sprache nicht kontextfrei ist, kann das **Pumping-Lemma** verwendet werden.

Satz: Pumping-Lemma

Jede reguläre Sprache L hat die folgende Pumping-Eigenschaft:

Es gibt eine Zahl $n \in \mathbb{N}_0$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvw$ geschrieben werden kann, so dass gilt:

- $|uv| \leq n$
- $|v| \geq 1$
- $\forall i \geq 0: uv^i w \in L$

Das bedeutet nach der klassischen Logik: Erfüllt eine Sprache die Pumping-Eigenschaft nicht, dann ist sie nicht regulär.

Ein Beispiel zum Pumping-Lemma: Zeige, dass $L = \{a^i b^i : i \in \mathbb{N}\}$ nicht regulär ist:

- Der Gegner wählt ein $n \in \mathbb{N}_0$ (also n beliebig)
- Wir wählen das Wort $z = a^n b^n \in L$ und offensichtlich gilt $|z| \geq n$.
- Der Gegner wählt eine Zerlegung von $z = uvw$ mit $|uv| \leq n$, $|v| \geq 1$.
- Dann ist oBdA $u = a^r$, $v = a^s$ mit $r + s \leq n$, $s > 0$ und $w = a^t b^n$ mit $r + s + t = n$.
Dann können wir $i = 2$ wählen und es gilt:

$$uv^i w = a^r a^{2s} a^t b^n = a^{n+s} b^n \notin L$$

1.1.2 Entscheidbarkeit bei Typ 1,2,3-Sprachen

Definition: Entscheidbarkeit

Eine Sprache heißt entscheidbar, wenn es einen Algorithmus gibt, der bei der Eingabe einer Grammatik G und einem Wort w in endlicher Zeit feststellt, ob $w \in L(G)$ oder nicht gilt.

Dabei sind alle Sprachen vom Typ 1,2 und 3 **entscheidbar**. Alternativ wird das auch das **Wortproblem** genannt. Weiter dazu siehe Kapitel 1.6.1

1.1.3 Deterministische Endliche Automaten

Satz: Reguläre Sprachen in Automaten

M ist ein DFA $\iff L(M)$ regulär.

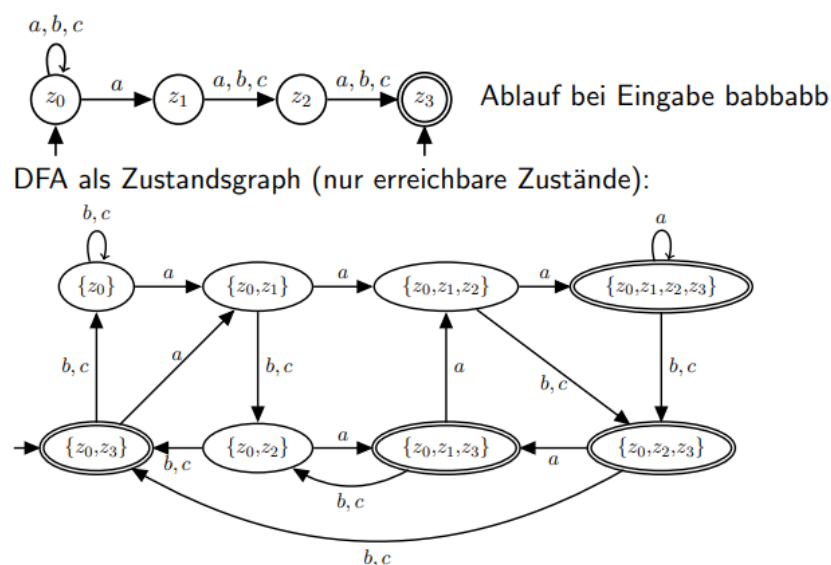
Für jede reguläre Sprache gibt es einen NFA, der sie akzeptiert (Rabin & Scott: Jede von einem NFA akzeptierte Sprache ist auch durch einen DFA akzeptierbar.) Ebenso akzeptieren NFAs mit ϵ -Übergängen genau die regulären Sprachen.

1.1.4 Potenzmengenkonstruktion

Über die Potenzmengenkonstruktion ist es möglich einen NFA in einen DFA zu überführen. Dabei sind:

- Die Zustände im DFA Mengen aus Zuständen des NFA.
- Jeder Zustand, der einen Endzustand des NFA enthält ist selber Endzustand.
- Die Startzustände werden in einem Startzustand gebündelt.

Vorgehen: Führe von jedem Knoten eine Verbindung zu einem Anderen, der genau alle erreichbaren Zustände enthält. Falls die Kombination noch nicht existiert, lege sie neu an und vervollständige zuerst alle nicht fertigen Knoten.



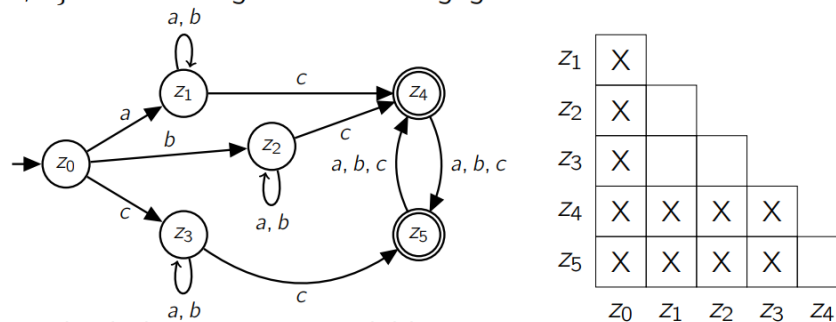
1.1.5 Minimierung eines DFAs

Hierbei geht es darum einen minimalen Automaten zu konstruieren. Dieser enthält dann nicht mehr Zustände als Knoten sondern Äquivalenzklassen von Zuständen.

Gehe dazu wie folgt vor.

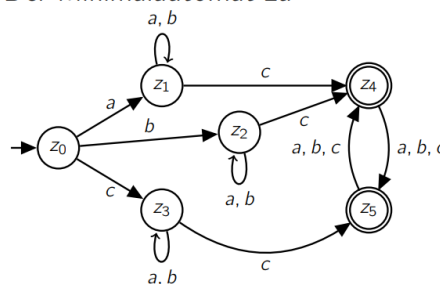
1. Entferne nicht erreichbare Zustände.
2. Zeichne die Tabelle wie im Beispielbild und markiere alle Tupel $\{z, z'\}$ für $z \notin E$ und $z' \in E$
3. Füge dann überall Markierungen zu Tupeln $\{z_i, z_j\}$ hinzu, für die gilt: $\exists a \in \Sigma : \{\delta(z_i, a), \delta(z_j, a)\}$ ist markiert.
4. Die leeren Felder sind Äquivalenzen von Zuständen.
5. Führe alle äquivalenten Zustände in eine Klasse zusammen und bilde aus den Klassen die neuen Knoten. Füge dann die Produktionen ein. Fertig.

Sei $\Sigma = \{a, b, c\}$ und der folgende DFA M gegeben:

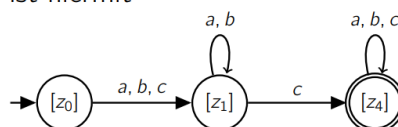


- ▶ alle Zustände sind von z_0 aus erreichbar
- ▶ äquivalente Zustände berechnen: Tabelle T erstellen
- ▶ Initiales Markieren: $\{z, z'\}$ mit $z \in \{z_0, z_1, z_2, z_3\}$ und $z' \in \{z_4, z_5\}$
- ▶ $\{z_0, z_1\}$, da $\{\delta(z_0, c), \delta(z_1, c)\} = \{z_3, z_4\}$ bereits markiert ist,
- ▶ $\{z_0, z_2\}$, da $\{\delta(z_0, c), \delta(z_2, c)\} = \{z_3, z_4\}$ bereits markiert ist, und
- ▶ $\{z_0, z_3\}$, da $\{\delta(z_0, c), \delta(z_3, c)\} = \{z_3, z_5\}$ bereits markiert

Der Minimalautomat zu



ist hiermit



1.1.6 Reguläre Ausdrücke

Definition: Reguläre Ausdrücke

Sei Σ ein Alphabet. Ein regulärer Ausdruck ist induktiv definiert:

- $\emptyset$ ist ein regulärer Ausdruck.
- ϵ ist ein regulärer Ausdruck.
- $a \in \Sigma$ ist ein regulärer Ausdruck.
- Sind a_1, a_2 reguläre Ausdrücke, dann sind es auch: $a_1 a_2$, $(a_1 | a_2)$ und $(a_1)^*$.

Dabei ist $(a_1 | a_2)$ die Auswahl.

Satz: Satz von Kleene

Reguläre Ausdrücke erzeugen genau die regulären Sprachen.

1.1.7 Satz von Myhill-Nerode

Der Satz ermöglicht eine genaue Definition der regulären Sprachen. Dafür benötigt man die Nerode-Relation:

Definition: Nerode-Relation

Sei L eine formale Sprache über Σ . Die Nerode-Relation ist definiert durch:

$$u \sim_L v \iff \forall w \in \Sigma^*: uw \in L \iff vw \in L$$

Satz: Myhill-Nerode

Eine formale Sprache L ist genau dann regulär, wenn der Index von $\sim_L$ endlich ist.

Beispiel: Die Sprache $L = \{ab^n : n \geq 0\}$ ist regulär, denn sie hat den Index 3:

- $[\epsilon]_{\sim_L} = \{\epsilon\}$
- $[a]_{\sim_L} = \{a, ab, abb, \dots\} = L$
- $[a]_{\sim_L} = \Sigma^* \setminus (L \cup \{\epsilon\})$

Beispiel: Gegenargument für Regularität: Die Sprache $L = \{a^k b^l c^l : k, l \in \mathbb{N}\} \cup \{b, c\}^*$ ist nicht regulär:

- Betrachte $[u_i]_{\sim_L} = [ab^i c^i]_{\sim_L}$ für $i \in \mathbb{N}$.
- Mit $w_i = c^{i-1}$ gilt: $u_i w_i = ab^i c^i \in L$ aber es gilt: $u_j w_i = ab^j c^i \notin L$ für $i \neq j$
- Damit sind alle diese Äquivalenzklassen disjunkt und der Index ist unendlich.
- Mit dem Satz von Myhill-Nerode ist die Sprache L nicht regulär.

Allgemeines zu regulären Sprachen

- Das Pumping-Lemma kann verwendet werden um die nicht-Regularität zu zeigen.
- Reguläre Ausdrücke erzeugen reguläre Sprachen.
- DFAs und NFAs ohne/mit ϵ -Übergängen erkennen genau die regulären Sprachen.
- Reguläre Sprachen sind unter Schnitt, Vereinigung, Komplement, Kleenschem Abschluss und Produkt abgeschlossen.
- Der Myhill-Nerode Index ist endlich $\iff$ reguläre Sprache.

1.2 Kontextfreie Sprachen

Im Allgemeinen gilt: Kontextfreie Sprachen sind abgeschlossen unter Vereinigung, Produkt und Kleenschem Abschluss. Jedoch unter Schnitt und Komplementbildung **nicht**.

1.2.1 Chomsky-Normalform

Definition: Chomsky-Normalform

Eine kontextfreie Grammatik ist in Chomsky-Normalform, wenn für alle Produktionen P erfüllt ist:

$$P: A \rightarrow w \in \Sigma \text{ oder } A \rightarrow BC \ (B, C \in V)$$

Für die Sprache einer kontextfreien Grammatik kann eine Grammatik in Chomsky-Normalform gefunden werden.

Pumping-Lemma für kontextfreie Sprachen

Satz: Pumping- für kontextfreie Sprachen

Jede kontextfreie Sprache L hat die folgende Pumping-Eigenschaft:
Es gibt eine Zahl $n \in \mathbb{N}_0$, sodass jedes Wort $z \in L$, welches Mindestlänge n hat (d.h. $|z| \geq n$), als $z = uvwxy$ geschrieben werden kann, so dass gilt:

- $|vwx| \leq n$
- $|vx| \geq 1$
- $\forall i \geq 0: uv^iwx^iy \in L$

Das bedeutet nach der klassischen Logik: Erfüllt eine Sprache die Pumping-Eigenschaft nicht, dann ist sie nicht kontextfrei.

Eine Sprache, die auf einem unären Alphabet ($|\Sigma| = 1$) definiert ist, ist genau dann kontextfrei, wenn sie regulär ist.

Daraus folgt, dass die folgenden Sprachen nicht kontextfrei sind (da sie nicht regulär sind):

- $L_1 := \{a^p : p \text{ Primzahl}\}$
- $L_2 := \{a^p : p \text{ keine Primzahl}\}$

- $L_3 := \{a^n : n \text{ Quadratzahl}\}$
- $L_4 := \{a^{2^n} : n \in \mathbb{N}\}$

1.2.2 Kellerautomaten

Kellerautomaten haben auch einen Zustandsautomat. Aber zusätzlich wird bei jedem Lesen eines Buchstaben auch der oberste Kellerinhalt gelesen und entsprechend der Produktion verändert und es wird in einen neuen Zustand gewechselt.

Satz: Kellerautomaten für kontextfreie Sprachen

Die kontextfreien Sprachen werden genau von den Kellerautomaten (mit/ohne Endzuständen) erkannt.

Ein Kellerautomat ist deterministisch, wenn für alle Übergänge $\delta(Z, \Sigma, \Gamma)$ gilt:

$$|\delta(z, a, A)| + |\delta(z, \epsilon, A)| \leq 1$$

DAs heißt, für jedes Wort hat der DPDA einen eindeutigen Lauf.

1.3 Das Wortproblem

Das Wortproblem allgemein ist die Frage, ob ein Wort w in der erzeugten Sprache $L(G)$ einer Grammatik G liegt.

1.3.1 Cocke-Younger-Kasami-Algorithmus

Dieser Algorithmus erlaubt es, das Wortproblem für ein Wort aus einer kontextfreien Sprache in Chomsky-Normalform in Polynomialzeit zu entscheiden.

Dazu wird eine linke obere Dreiecksmatrix gezeichnet, mit $|w| = i$ Spalten Zeilen. Im Eintrag $V(i, j)$ wird geprüft, ob es eine Ableitung des Teilworts der Länge j ab dem i -ten Buchstaben gibt. Prüfe dazu die Produktionsmöglichkeiten oberhalb des Eintrags. Gehe jede Zeile darüber durch und finde den dazu passenden Eintrag im Rest der Matrix. Also um $V(3, 4)$ zu bestimmen finde eine Produktion, die das Teilwort der Länge 4 ab Stelle 3 produziert. Dafür kann man durchgehen: $V(3, 1)V(4, 3)$, $V(3, 2)V(5, 2)$ und $V(3, 3)V(6, 1)$. Gibt es eine passende Produktion, trage sie ein. Bei mehreren, trage alle ein.

Paul, hier bitte den Algorithmus bildlich einfügen

1.4 Kontextsensitive Sprachen

Satz: Satz von Kuroda

Die kontextsensitiven Sprachen werden von den nichtdeterministischen linear beschränkten Turingmaschinen erkannt.

Allgemein werden alle Typ-0 Sprachen genau von den nichtdeterministischen Turingmaschinen erkannt. Dabei spielt die lineare Beschränktheit erst für die Laufzeiten eine Rolle. Der Satz von Immermann besagt, dass die kontextsensitiven Sprachen abgeschlossen unter Komplementbildung sind.

1.5 Überblick Sprachtypen

Matthias, bitte hier die Übersicht aus VL8 einfügen!

1.6 Berechenbarkeit und Reduktionen

Definition: Turingberechenbarkeit

Eine Funktion $f: \Sigma^* \rightarrow \Sigma^*$ ist turing berechenbar, wenn gilt:

$$f(u) = v \iff z_0 u \vdash^* \square \dots \square z_e v \square \dots \square, \quad z_e \in E$$

Die LOOP-Programme sind immer totale Funktionen. Es gibt turingberechenbare Funktionen, die aber nicht LOOP-Berechenbar sind (zum Beispiel die überall undefinierte Funktion).

Addition, Multiplikation und andere Rechenarten sind LOOP-Berechenbar. Auch IF-Programme sind LOOP-berechenbar.

LOOP-berechenbare Funktionen sind auch WHILE-berechenbar und WHILE-Berechenbare Funktionen sind auch Turing-Berechenbar.

$$\text{Turing-berechenbar} \iff \text{WHILE-berechenbar} \iff \text{GOTO-berechenbar}$$

Die Ackermann-Funktion ist ein klinisches Beispiel für eine totale Funktion, die WHILE-berechenbar ist (zum Beispiel, weil sie leicht in einer modernen Programmiersprache zu implementieren ist), aber nicht LOOP-berechenbar ist.

Die primitiv rekursiven Funktionen werden genau von LOOP-Programmen berechnet. Die μ -rekursiven Funktionen entsprechen genau den WHILE-berechenbaren Funktionen.

1.6.1 Entscheidbarkeit

Definition: Entscheidbarkeit

Eine Sprache $L \subseteq \Sigma^*$ heißt entscheidbar, wenn die charakteristische Funktion von L :

$$\chi_L: \Sigma^* \rightarrow \{0, 1\}, \quad \omega \mapsto \begin{cases} 1 & \omega \in L \\ 0 & \omega \notin L \end{cases}$$

berechenbar ist.

Dazu eng verwandt ist die semi-Entscheidbarkeit: Eine Sprache heißt **semi-entscheidbar**, wenn die charakteristische Funktion für $\omega \notin L$ undefiniert ist. Die semi-entscheidbaren Sprachen sind die rekursiv aufzählbaren Sprachen.

Satz: Kriterium für Entscheidbarkeit

Eine Sprache L ist genau dann entscheidbar, wenn L und $\bar{L}$ semi-entscheidbar sind.

Eine Sprache L ist genau dann entscheidbar, wenn $\bar{L}$ entscheidbar ist.

Satz: Spezielles Halteproblem

Das spezielle Halteproblem ist die Sprache

$$K := \{\omega \in \{0, 1\}^* \mid M_\omega \text{ hält für die Eingabe } \omega\}$$

und ist **nicht** entscheidbar.

1.6.2 Reduktionsbeweise

Dass eine Sprache entscheidbar oder unentscheidbar ist, ist häufig nicht einfach von Grund auf zu Beweisen. Dafür gibt es die Reduktion:

Definition: Reduktion

Seien $L_1 \subseteq \Sigma_1^*$ und $L_2 \subseteq \Sigma_2^*$ Sprache. Dann sagen wir L_1 ist auf L_2 reduzierbar, geschrieben $L_1 \leq L_2$, falls es eine totale und berechenbare Funktion $f: \Sigma_1^* \rightarrow \Sigma_2^*$ gibt, sodass für alle $\omega \in \Sigma_1^*$

$$w \in L_1 \iff f(\omega) \in L_2$$

Satz: Reduktionsbeweis

Wenn $L_1 \leq L_2$ und L_2 entscheidbar ist, dann ist auch L_1 entscheidbar.

Die Kontraposition: Wenn $L_1 \leq L_2$ und L_1 ist unentscheidbar, dann ist auch L_2 unentscheidbar.

Satz: Allgemeines Halteproblem

Das allgemeine Halteproblem ist die Sprache

$$H := \{\omega \# x \mid M_\omega \text{ hält für die Eingabe } x\}$$

Es ist **nicht** entscheidbar.

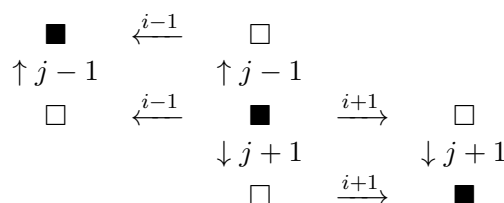
Das Halteproblem auf leerem Band H_0 ist unentscheidbar.

Satz: von Rice

Sei $\mathfrak{R}$ die Menge der Turingberechenbaren Funktionen. Sei $\mathfrak{S}$ eine Teilmenge $\emptyset \subset \mathfrak{S} \subset \mathfrak{R}$. Dann ist

$$C(\mathfrak{S}) = \{\omega \mid \text{Die von } M_\omega \text{ berechnete Funktion liegt in } \mathfrak{S}\}$$

eine unentscheidbare Sprache.



2 Algorithmen und Datenstrukturen

Groß-O Notation, Typen von Listen (einfach, doppelt verkettet), Sortiervverfahren und deren Laufzeiten/Eigenschaften, Bäume, Operationen auf Bäumen, Suchverfahren und Eigenschaften incl Laufzeiten, Dijkstra, Jarnik-Prim, Kürzeste Wege, minimale Spannbäume, Hashing, Streuspeicherung

2.1 Algorithmen Allgemein

2.1.1 Eigenschaften von Algorithmen

Im Folgenden finden sich einige wichtige Eigenschaften von Algorithmen:

- Allgemeinheit (Lösung für eine Klasse von Problemen)
- Determiniertheit (gleiche Eingabe $\implies$ gleiche Ausgabe)
- Determinismus (gleicher Lauf bei gleicher Eingabe)
- Terminierung (ein Algorithmus braucht nur endlich viele Schritte)
- partielle Korrektheit (im Fall der Terminierung wird immer die spezifizierte Ausgabe berechnet)

2.2 Listen und Operationen auf Listen

Arten von Listen

Für Listen gibt es klassischerweise einige Beispiele. Das sind:

- Einfache Arrays
- Einfach verkettete Listen (Jeder Knoten kennt nur seinen Nachfolger)
- Doppelt verkettete Listen (Jeder Knoten kennt Vorgänger und Nachfolger)
- Einfach verankerte Listen (Start bekannt, Rest einfach verkettet)
- Doppelt verankerte Listen (Start und Ende bekannt, Rest einfach verkettet)
- Stacks (nach dem LIFO-Prinzip): Es kann immer nur auf das oberste Objekt zugegriffen werden, der Rest ist wieder einfach verkettet
- Queue (nach dem FIFO-Prinzip): Es kann nur am Ende eingefügt werden und auf den Anfang zugegriffen werden, der Rest ist einfach verkettet

2.2.1 Sortiervverfahren

Die gängigen Sortiervverfahren können den folgenden Komplexitätsklassen zugeordnet werden:

- $\mathcal{O}(n^2)$: BubbleSort, SelectionSort, InsertionSort
- $\mathcal{O}(n \log(n))$: MergeSort, QuickSort, HeapSort

In der Vorlesung wurde gezeigt:

Satz: Untere Schranke von Sortieralgorithmen

Die Anzahl der benötigten Vergleiche in jedem vergleichsbasierten Sortierv erfahren wächst im Worst-Case mindestens wie $n \log(n)$.

BubbleSort

Die Idee ist, dass die großen Einträge wie Luftblasen immer weiter aufsteigen. Dabei wird die Liste immer von vorne durchgegangen aber immer einen Schritt kürzer, da im ersten Schritt ja das größte Element bis ganz nach oben gestiegen ist. Somit bleibt die rechte Seite des Arrays sortiert.

SelectionSort

Die Idee ist, dass der erste Eintrag als Schlüssel gespeichert wird und dann das Minimum der rechten Seite gesucht wird. Dann wird der Schlüssel mit dem Minimum vertauscht oder gleich gelassen, wenn der Schlüssel das Minimum war. Danach wird das nächste Element als Schlüssel gewählt und wieder das neue Minimum gesucht. Somit bleibt die linke Teilliste immer sortiert.

InsertionSort

Die Idee ist, dass die Liste von einem Startelement nach rechts durchgegangen wird, bis ein Element gefunden wird, das die Ordnung verletzt. Dieses wird so lange nach vorne verglichen, bis der richtige Platz gefunden ist. Dann wird der Laufindex um eins erhöht und die Schleife beginnt erneut. Also wird die linke Teilliste immer sortiert gehalten.

Die Laufzeit ist im Average-Case und Worst-Case in $\mathcal{O}(n^2)$ und im Best-Case $\mathcal{O}(n)$ bei vorsortierter Liste.

MergeSort

Hier wird die Strategie Divide-and-Conquer angewandt: Zuerst werden die Listen in der Mitte geteilt und dann MergeSort auf die beiden Teillisten angewandt. Die Listen werden so lange zerstückelt, bis atomare Listen vorliegen. Diese sind sortiert. Dann werden die Listen mithilfe von Vergleichen sortiert zusammengefügt. Das passiert in allen Schritten wieder rückwärts und die Liste ist sortiert.

QuickSort

Die Idee ist, dass immer ein Element als Pivotelement gewählt wird. Gerne das letzte Element. Dann werden alle Elemente $\leq$ in die linke Teilliste gebracht und der rest in die rechte. Einelementige Listen sind bereits sortiert.

Alle Teillisten werden wieder mit Quicksort sortiert. Das Zusammenfügen ist dann trivial. Im Worst-Case ist Quicksort nicht schneller als die anderen Verfahren, es liegt in $\mathcal{O}(n^2)$. Die Wahl des Pivot-Elements ist eine Wissenschaft für sich und für fast jede Wahl kann ein Worst-Case Szenario gebaut werden. Jedoch ist meistens Median-Of-3 das schnellste.

2.2.2 Suchalgorithmen

Lineare Suche

Die Suche wird auch als sequenzielle Suche bezeichnet. Es wird die Liste von vorne her durchgegangen bis das entsprechende Element gefunden ist.

Die Laufzeit liegt im Worst-Case in $\mathcal{O}(n)$, weil die Liste einmal durchgegangen werden muss.

Binäre Suche

Diese Form funktioniert nur auf sortierten Listen. Es wird zur Mitte gesprungen und verglichen ob der Schlüssel kleiner, gleich oder größer ist. Dann wird entweder nach links oder rechts gesprungen oder das Element ausgegeben.

Bei jedem Schleifendurchlauf halbiert sich der zu durchsuchende Bereich, die Komplexität liegt also in $\mathcal{O}(\log(n))$. Sortieren davor erhöht die Laufzeit.

2.3 Bäume

2.3.1 Eigenschaften in Binärbäumen

Ein Binärbaum t der Höhe h hat die folgenden Eigenschaften:

- t hat maximal $2^{h+1} - 1$ Knoten.
- t hat mindestens $2^{h-1} + 1$ Knoten.
- t hat maximal $2^h - 1$ innere Knoten.
- t hat maximal 2^h Blätter

Ein Binärbaum kann in ein Array eingebettet werden. Diese wird erreicht durch das Ein speichern der Knoten von links nach rechts, gestaffelt in den Höhen. Also sind die Elemente aus der Höhe 2 (Es sind $2^{2-1} = 2$ Elemente) an den Arrayindizes 2 und 3 zu finden (An den Stellen 2^{h-1} bis $2^h - 1$).

2.3.2 Baumtraversierungen

Es werden üblicherweise drei Methoden verwendet:

- **Preorder:** Knoten - linker Teilbaum - rechter Teilbaum
- **Inorder:** linker Teilbaum - Knoten - rechter Teilbaum [**Mehrdeutigkeit möglich**]
- **Postorder:** linker Teilbaum - rechter Teilbaum - Knoten
- **Breitendurchlauf:** Ebenenweise durchlaufen (wie bei der Arrayeinbettung)

2.3.3 Binäre Suchbäume

Ein binärer Suchbaum erleichtert das Suchen. Ist der Knoten selbst nicht das gesuchte Element, so kann nach links gegangen werden um alle kleineren Elemente zu betrachten und nach rechts um alle größeren zu durchsuchen.

Beim **Einfügen** eines Schlüssels wird eine Suche nach dem Element gestartet und das Element dort eingefügt, an der die Suche nicht weiterkommt. Somit wird der neue Schlüssel ein Blatt.

Beim **Entfernen** eines Schlüssels wird dieser gesucht. Ist er nicht vorhanden passiert nichts. Ist er ein Blatt, so wird dieses entfernt. Ist es ein innerer Knoten, so wird er mit dem rechtesten Blatt des linken Teilbaums vertauscht und dann entfernt.

Laufzeiten in binären Suchbäumen

Die Best-Case Laufzeit liegt bei $\mathcal{O}(\log(n))$ und im Worst-Case bei $\mathcal{O}(n)$. Das liegt daran, dass maximal die ganze Baumtiefe durchsucht werden muss. Das kann im schlimmsten Fall eine lineare Liste sein und im besten Fall ein ausbalancierter Baum.

2.3.4 Varianten von Binärbäumen

Für bessere Suchergebnisse werden noch zwei weitere Arten von Binärbäumen verwendet:

- **AVL-Bäume** sind Binärbäume, die die Struktureigenschaft haben, dass jeweils beide Teilbäume eines Knotens sich in der Höhe nur um maximal 1 unterscheiden. Damit werden immer schnelle Suchen garantiert.

Zeitaufwand entsteht dadurch, dass Rotiert und eingefügt werden muss. Der Gesamtaufwand liegt bei $\mathcal{O}(\log(n))$.

Beim Einfügen in den Baum benötigt es höchstens eine (Doppel-)Rotation. Suche dafür den tiefsten Knoten, der nicht dem AVL-Kriterium der Tiefendifferenz entspricht: Wurzel mit rechtem Knoten vertauschen. Linken Teilbaum des rechten Knotens der Wurzel die jetzt links ist als rechten Teilbaum geben. Die restlichen Teilbäume können so bleiben wie sie sind. Alternativ geht das ganze auch gespiegelt.

- **Splay-Bäume** sind selbstoptimierende Bäume. Nach einer Suche wird der Schlüssel an die Wurzel rotiert. Dabei ist das Laufzeitverhalten asymptotisch die eines optimalen Baums. Es wird auch weniger Speicher verbraucht, da entgegen dem AVL-Baum keine Daten über die Tiefe etc. gespeichert werden müssen. Die Laufzeit selber ist $\mathcal{O}(\log(n))$.
- **Mehrwegebäume** haben immer einen Wechsel aus Schlüsseln und Zeiger auf weitere Teilbäume, die wiederum Mehrwegebäume oder Listen sein können. Darin kann wie in jedem Baum davor gesucht werden. Da die Blätter aus Listen bestehen können dort auch die klassischen Suchverfahren eingesetzt werden.

2.4 Laufzeiten

Für die Notation von Laufzeiten wird die Landau-Notation verwendet. Dabei ist eine Funktion in einer Laufzeitklasse (geschrieben $f \in \mathcal{O}(\dots)$), wenn gilt:

$$\mathcal{O}(f) = \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \exists c > 0 : \exists x_0 > 0 : \forall x \geq x_0 : |g(x)| \leq c \cdot |f(x)|\}$$

Das heißt also, alle Funktionen einer Laufzeitklasse $\mathcal{O}(f)$ wachsen höchstens so schnell wie f . Für die Informatik werden häufig die reellen Zahlen durch die natürlichen Zahlen ersetzt.

$$o(f) = \{g : \mathbb{R} \rightarrow \mathbb{R} \mid \forall c > 0 : \exists x_0 > 0 : \forall x \geq x_0 : |g(x)| \leq c \cdot |f(x)|\}$$

Alternativ können die Klassen auch mithilfe des Limes definiert werden:

$$o(f) = \left\{ g: \mathbb{R} \rightarrow \mathbb{R} : \lim_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| = 0 \right\}$$

$$\mathcal{O}(f) = \left\{ g: \mathbb{R} \rightarrow \mathbb{R} : \limsup_{x \rightarrow \infty} \left| \frac{f(x)}{g(x)} \right| < \infty \right\}$$

Es gibt noch weitere Arten von Komplexitätsklassen:

$$f \in \Omega(g) \iff \exists c > 0 : \exists x_0 > 0 : \forall x > x_0 : c \cdot |g(x)| \leq |f(x)|$$

$$f \in \Theta(g) \iff \exists c > 0 : \exists C > 0 : \exists x_0 > 0 : \forall x > x_0 : c \cdot |g(x)| \leq |f(x)| \leq C \cdot |g(x)|$$

Rechenregeln für die Landau-Notation

- $f + g \in \mathcal{O}(\max(f, g))$
- $f \cdot g \in \mathcal{O}(f \cdot g)$
- $g \in \mathcal{O}(f) \iff a \cdot g \in \mathcal{O}(f)$
- Logarithmen unterschiedlicher Basen fallen in die gleiche Klasse

Rekursionsformeln

Für die Regel Divide-and-Conquer gilt die folgende Eigenschaft zur Laufzeit:
Für eine Rekursionsgleichung der Form

$$T(n) = \begin{cases} c & n \leq 1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & n > 1 \end{cases}$$

mit $c > 0, a > 0, b > 1, f(n) \in \mathcal{O}(n^d), d > 0$ gilt:

$$T(n) \in \begin{cases} \mathcal{O}(n^d) & d > \log_b(a) \\ \mathcal{O}(n^d \log(n)) & d = \log_b(a) \\ \mathcal{O}(n^{\log_b(a)}) & d < \log_b(a) \end{cases}$$

Für die Regel Subtract-and-Conquer gilt die folgende Eigenschaft zur Laufzeit:

$$T(n) = \begin{cases} c & n \leq 1 \\ a \cdot T(n - b) + f(n) & n > 1 \end{cases}$$

mit $c > 0, a > 0, b > 1, f(n) \in \mathcal{O}(n^d), d \geq 0$ gilt:

$$T(n) \in \begin{cases} \mathcal{O}(n^d) & a < 1 \\ \mathcal{O}(n^{d+1}) & a = 1 \\ \mathcal{O}(n^d a^{\frac{n}{b}}) & a > 1 \end{cases}$$

2.5 Hashing

Ziel ist es eine Schlüsselmenge auf eine (relativ) kleine Liste zu schreiben. Dafür benötigt man eine Hashfunktion. Diese muss zumindest surjektiv sein.

2.5.1 Umgang mit Kollisionen

Bei einer Kollision beim Einfügen müssen Lösungen der Kollision gefunden werden. Dabei wird in zwei Kategorien unterschieden:

- **Offenes Hashing mit geschlossener Adressierung:** Hier wird bei einer Kollision die Adresse mit einer Datenstruktur verknüpft, die eine Suchfunktion bietet. Je nach Methode wird auch direkt jeder Hashwert mit einer solchen Struktur verknüpft.
- **Geschlossenes Hashing mit offener Adressierung:** Hierbei wird bei der Kollision eine Sondierungsfunktion verwandt. Die Hashtabelle wird weiter durchgegangen und es wird bis zum nächsten freien Platz sondiert.

2.5.2 Gängige Sondierungen

- **Lineares Sondieren:** Hierbei wird nach einer Kollision der nächste freie Platz in c_1 Schritten gesucht:

$$h(x, j) = (h(x) + c_1 j) \bmod m$$

- **Quadratisches Sondieren:** Hierbei wird nach einer Kollision in quadratischer Schrittweite ein neuer Platz gesucht:

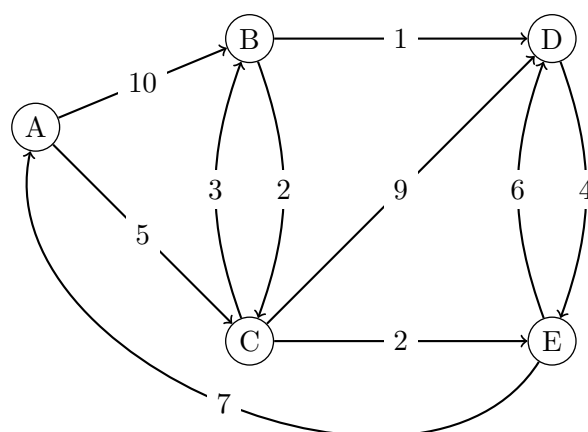
$$h(x, j) = (h(x) + c_2 j^2)$$

- **Ein Versuch für optimales Sondieren:** Hierbei werden zwei Hashfunktionen h und h' verwandt, deren Kollisionswahrscheinlichkeit bei $\frac{1}{m}$ liegt (also dass $\mathbb{P}(h(x) = h(y)) = \frac{1}{m} = \mathbb{P}(h'(x) = h'(y))$ liegt, dabei sind die x in beiden Vorkommen gleich genauso wie die y). Die Sondierung lautet dann

$$h(x, k) = (h(x) + h'(x)j^2)$$

2.6 Algorithmus von Dijkstra

NACHTRAGEN!



k	W_k	S_k	$D_k(B)$	$D_k(C)$	$D_k(D)$	$D_k(E)$
0	—	$\{A\}$	10	5	∞	∞
1	C	$\{A, C\}$	8	5	14	7
2	E	$\{A, C, E\}$	8	5	13	7
3	B	$\{A, C, E, B\}$	8	5	9	7
4	D	$\{A, C, E, B, D\}$	8	5	9	7

2.7 Minimale Spannbäume

Ein Minimaler Spannbaum ist ein ungerichteter Subgraph eines ungerichteten Graphen. Jeder Knoten ist enthalten und die Kantensumme ist minimal. Für die Berechnung gibt es zwei Methoden:

Prim-Algorithmus

Starte bei einem beliebigen Knoten. Dann wird von diesem Knoten ausgehend die minimale Kante hinzugefügt. Dieser Schritt wird immer für den gesamten Subgraph wiederholt. Finde von allen hinzugefügten Knoten die dann minimale Kante hinzu, die einen noch nicht erreichten Knoten hinzufügt. So bleibt die Zyklenfreiheit erhalten. Die Laufzeit liegt in $\mathcal{O}(|V|^2)$

Algorithmus von Kruskal

Zuerst werden die Kanten der Länge (des Gewichts) nach sortiert. Dann werden die Kanten schrittweise dem minimalen Spannbaum hinzugefügt. Eine Kante wird nicht hinzugefügt, falls der neu verbundene Knoten bereits im Spannbaum enthalten ist. Somit ist die Zyklenfreiheit und die minimale Kantensumme gewährleistet. Die Laufzeit liegt in $\mathcal{O}(|E| \cdot \log(|E|))$

2.8 Algorithmische Paradigmen

Backtracking

Idee:

- Lösung eines Problems durch Versuch und Irrtum (trial and error) .
- Schrittweises „Herantasten“ an die Gesamtlösung.
- Kann eine Teillösung nicht zu einer Gesamtlösung erweitert werden:
 - Letzten Schritt rückgängig machen.
 - Weitere, alternative Schritte probieren.

Voraussetzung:

- Die Lösung setzt sich aus „Komponenten zusammen“
- Für jede Komponente gibt es mehrere Wahlmöglichkeiten
- Teillösungen können getestet werden.

Geeignete Probleme:

- Damenproblem

- Sudoku
- Solitär-Brettspiel
- Wegsuche von A nach B

Greedy-Algorithmen

Bei Greedy-Algorithmen wird aus dem aktuellen Lösungszustand immer der Nächste gewählt, der für den Moment der nächstbeste erscheint. Ein Beispiel dafür ist der Prim-Algorithmus.

Das Problem daran ist, dass diese Algorithmen nicht immer die optimale Lösung finden. Jedoch liefern sie meist eine schnelle und relativ gute Lösung.

Dynamische Programmierung

Die dynamische Programmierung versucht erst kleiner Probleme zu lösen. Diese Lösungen sollen dann eine Hilfe oder Grundlage für das nächstgrößere Problem sein.

Ein Beispiel kann die Lösung des Knapsack-Problems sein: Versuche zuerst eine Lösung für nur ein Objekt, dann zwei Objekte, usw zu finden.

3 Softwaresysteme

3.1 Projektplanung

3.1.1 Softwaremaße

Um ein SW-Projekt zu bemessen gibt es verschiedene Arten zu messen. Dabei gibt es drei bekannte Maße:

- **Größenorientierte Maße (LOC/NCSS):** Hierbei wird die Anzahl der Codezeilen gemessen. Probleme: Wann arbeitet man viel? Wie wird die Qualität sichergestellt? Wie lang sind Anweisungen in verschiedenen Programmiersprachen?
- **Funktionsorientierte Maße:** Versuch, die Komplexität aus der Funktionalität abzuleiten. Wie viele verschiedene Funktionen hat eine Software. Problem: Sie ist stark abhängig von der Qualität des Entwurfs oder der Anforderungen und kann erst spät angeandt werden.
- **McCabe:** Sie wird auch **zyklomatische Komplexität** genannt. Dazu werden die Module auf ihre Komplexität hin untersucht. Dafür wird der Kontrollflussgraph gezeichnet und die Komplexität entspricht:

$$V(Z) = |E| - |V| + 2$$

Es fällt auf, dass beispielsweise Sequenzen die Komplexität nicht erhöhen. Neue Schleifen hingegen schon.

- **Objektorientierte Maße:** Ableitung der Komplexität aus der Objektstruktur. Probleme: Wie größenorientiert und kann bei vielen Sprachen nicht angewandt werden.

3.1.2 Qualitätskriterien von Code

- **Korrektheit**
- **Wartbarkeit**
- **Integrität:** Wahrscheinlichkeit einer erfolgreichen Attacke.
- **Benutzbarkeit**

3.1.3 GANTT-Diagramme

In diesen Diagrammen werden die Arbeitspakete mit Boxen eingetragen. Die Länge der Box entspricht der geschätzten Dauer. Alle Dependencies sind mit Pfeilen gekennzeichnet. Ein Paket kann frühestens eingetragen werden, wenn alle Dependencies gelöst sind. Es werden so viele Mitarbeiter benötigt, wie es maximal gleichzeitig laufende Pakete gibt.

Die **kritischen Pfade** sind solche, die die gesamte Dauer eines Projekts umspannen. Ein Beispiel für ein GANTT-Diagramm gibt es [hier](#).

3.1.4 PERT/CPM Netzwerke

In einem CPM-Netzwerk werden die Anfänge oder Enden eines Projekt-Meilensteins verzeichnet. Die Kanten tragen die Dauer um die Aktivität zu machen. Zusätzlich werden in den Knoten die früheste und späteste Startzeit/Endzeit eingetragen. Dependencies ohne Zeitaufwand werden gestrichelt dargestellt.

Um die früheste Startzeit zu berechnen, werden die Abhängigkeiten und benötigten Zeiten von vorne durchgerechnet. Um die späteste zu berechnen, werden die benötigten Zeiten von hinten durchgerechnet. Dadurch erkennt man kritische Pfade. Knoten auf einem kritischen Pfad sind solche, die eine gleiche früheste und späteste Startzeit haben.

3.2 Testen

Beim Testing gibt es zwei grundsätzlich unterschiedliche Verfahren:

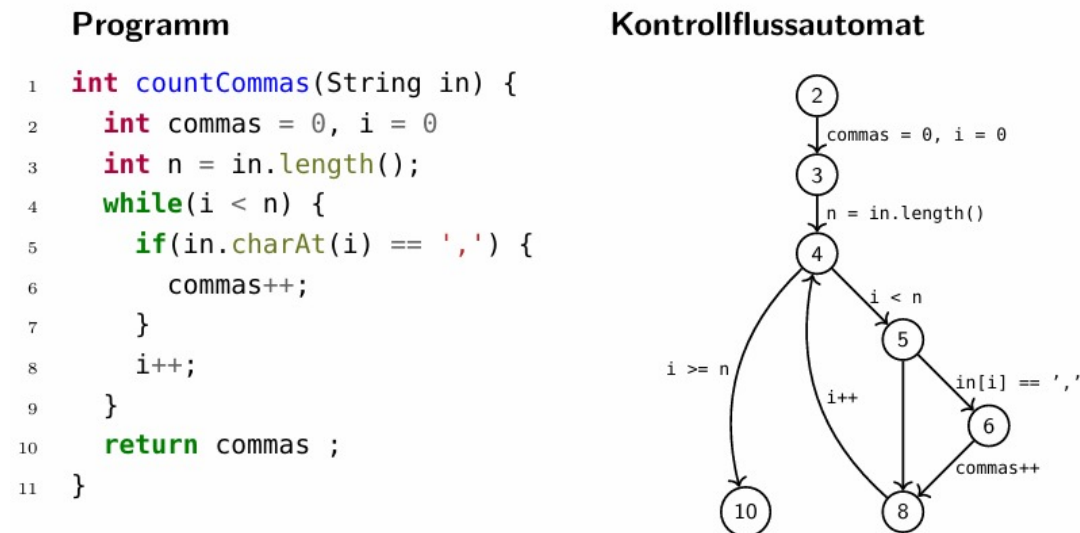
- **White-Box Testing:** Die Testfälle werden anhand des vorliegenden Codes generiert und durchgeführt.
- **Black-Box Testing:** Es ist nur bekannt was der zu testende Code machen soll und welche Eingaben er annehmen soll. Aufgrund dieser Wissenslage werden dann Testfälle generiert.

3.2.1 Überdeckungsmaße

- **statement coverage / Anweisungsüberdeckung:** Wurden alle Knoten/Anweisungen einmal besucht?
- **branch coverage / Zweigüberdeckung:** Wurden alle Fallunterscheidungen genommen?
- **path coverage / Pfadüberdeckung:** Wurden alle möglichen Pfade durch das Programm ausgeführt (meistens unendlich viele...daher nicht so geeignet)

3.2.2 Kontrollflussgraphen (CFG)

Ein Kontrollflussgraph ist eine Repräsentation der möglichen Wege in einem Programm(abschnitt). Dabei sind die Codezeilen Knoten und die möglichen Anweisungen und Tests sind die Kanten. Die Zyklen in einem CFG entsprechen den Schleifen in einem Programm. Das Beispiel stammt aus der Vorlesung FSV.



3.3 Prozessmodelle

3.3.1 Wasserfallmodell

Das Wasserfallmodell beschreibt eine mögliche Herangehensweise an ein Softwareprojekt. Es läuft ab in 5 Schritten:

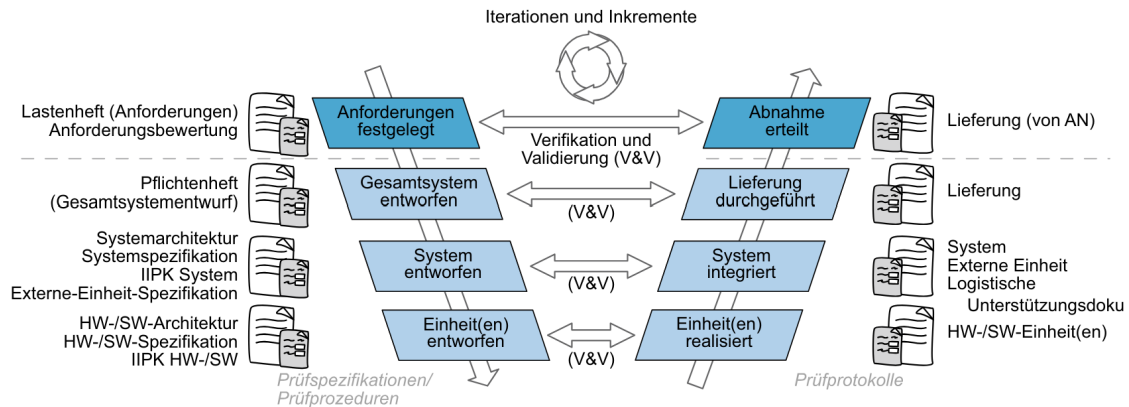
1. Anforderungsdefinition (Lasten- und Pflichtenheft)
2. System- und Softwareentwurf (Entwurfsspezifikation)
3. Programmierung und Unit-Testing (Programmcode und JavaDoc)
4. Integration und Systemtest (Testdokumentation und Berichte)
5. Betrieb und Wartung (Handbuch)

Vorteile des Wasserfallmodells sind, dass Festpreiverträge möglich sind. Zusätzlich kann der Fortschritt des Projekts leicht überwacht werden, es ist sehr effizient und kann leicht verstanden werden.

Die **Nachteile** des Wasserfallmodells sind, dass es häufig sehr lange Lastenhefte gibt, da alle Funktionalitäten eingefroren sind nach dem ersten Schritt. Die abstrakte Beschreibung ist meist sehr schwierig für den Kunden. Außerdem wird bei Kürzungen im Budget oder bei einer Verlängerung eventuell nichts ausgeliefert und der Kunde weiß erst am Ende, was er überhaupt für eine Software bekommt. Das Lernen findet erst nach dem Projekt statt.

3.3.2 V-Modell

Das V-Modell ist auch ein sequenzielles Modell, wobei hier die Phasen gegenüber gekoppelt sind (beim Wasserfallmodell nur die jeweils benachbarten).



3.3.3 Evolutionäre Methoden

Hierbei wächst das System von einer kleinen Aufgabe zu einem immer größeren System heran. Die Schritte entscheidet eher der Kunde. Dabei ist eine Methode recht bekannt: **Prototyping**.

Das Vertikale Prototyping hat zum Ziel einen Durchstich durch das Produkt zu erstellen. Ein funktionaler Ausschnitt des Systems soll durch alle Ebenen hindurch entwickelt sein. Das Horizontale Prototyping hat zum Ziel eine Ebene des Produkts vollständig zu entwickeln.

Dabei sind die Vorteile, dass die Kommunikation der Anforderungen leicht ist. Ansonsten analog zum Wasserfallmodell. Die Nachteile sind ebenfalls die des Wasserfallmodells und zum Beispiel, dass Kosten meist falsch eingeschätzt werden und die Anforderungen und Ausführung des Prototypen sehr schlampig sein können.

3.3.4 Spiralmodell

Das Spiralmodell ist eine Spezialisierung des iterativen Modells. Es werden vier Quadranten spiralförmig durchlaufen:

1. Festlegung von Zielen, Alternativen, Rahmenbedingungen
2. Evaluieren von Alternativen und Risikoabschätzung
3. Realisierung des Zwischenzyklus
4. Planung des nächsten Zyklus

3.3.5 Das agile Manifest

Das agile Manifest kennt 4 Leitsätze und 12 Prinzipien. Die Leitsätze lauten:

1. Individuum vor Prozess
2. Funktion vor Dokumentation
3. Mehr Zusammenarbeit mit dem Kunden als Vertragsverhandlungen

4. Reagieren auf Veränderungen als Festhalten an Plänen

Die Prinzipien lauten:

1. Frühe und kontinuierliche Auslieferung
2. Veränderungen sind immer (auch spät) willkommen
3. Funktionsfähige Produkte werden ausgeliefert
4. Experten und Entwickler arbeiten täglich zusammen
5. Individuen müssen motiviert sein
6. Gespräche sind das A&O
7. Das Fortschrittsmaß sind funktionierende Produkte
8. Gleiches Tempo auf unebgrenzte Zeit haltbar
9. Gutes Design und technische Exzellenz
10. Einfachheit
11. selbstorganisierte Teams
12. Reflektionen in den Teams

3.3.6 Methoden und Praktiken

Ein kleiner Überblick über die möglichen Methoden in agilen Projekten:

- Pair-Programming
- Test-Driven-Development
- Refactoring (möglich für Refactoring: Lesbarkeit, Erweiterbarkeit, Testbarkeit, Entfernung toten Codes, ...)
- DevOps (Experten und Programmierer arbeiten zusammen, da Entwicklung und Operation zusammenwirken müssen)

3.4 SCRUM

3.4.1 Artefakte

Die drei Artefakte des SCRUM sind:

- Der **Product-Backlog** enthält alle Features die eine Software haben soll. Er wird vom Productowner verwaltet.
- Der **Sprint-Backlog** enthält alle Features, die innerhalb eines Sprints fertig gestellt werden sollen. Er wird durch das Entwicklungsteam am Anfang eines Sprints befüllt.
- Das **Increment** ist das Produkt, das zum aktuellen Entwicklungsstand ausgeliefert werden kann.

3.4.2 Rollen

- Der **Produktinhaber** ist der Auftraggeber und er entscheidet welche Features und in welcher Reihenfolge diese entwickelt werden. Er beantwortet Fragen der Entwickler und hat die Aufsicht.
- Das **Entwicklungsteam** ist für die Software, die Dokumentation und die Tests zuständig. Sie sind selbstorganisiert.
- Der **SCRUM-Master** kommuniziert die Werte, Prinzipien und Praktiken und ist verantwortlich für den Prozess. Er passt die Prozesse an das Produkt an. Er ist nicht der Projektmanager oder Personaler.

3.4.3 Ablauf

Ein Projekt das mit SCRUM gemanaged wird läuft in drei Phasen ab:

1. Überblicksplanung
2. Sprints
3. Projektabschluss

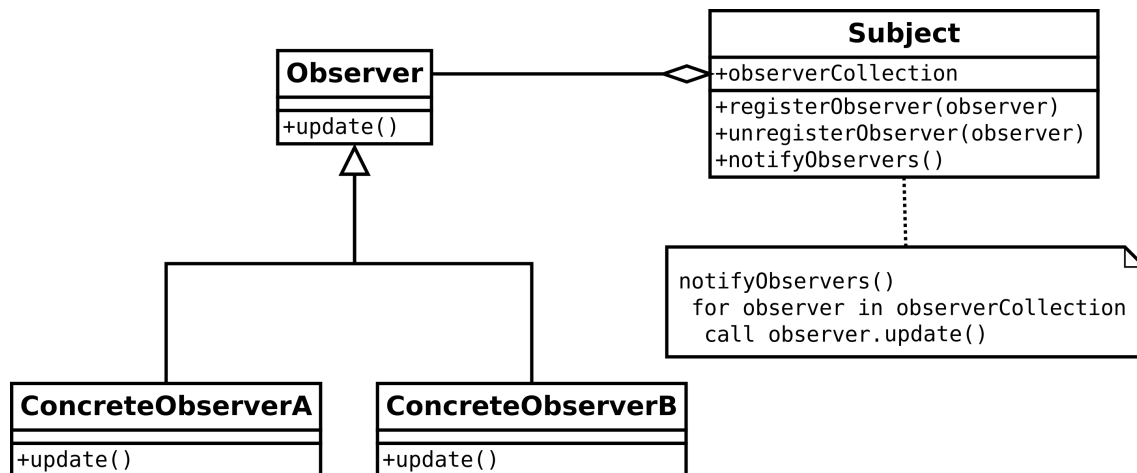
3.5 Designpatterns

Architekturpattern wichtig fürs Systemdesign (wie ist das System aufgebaut? Zum Beispiel MVC, Server-Client (evtl mit API, Beschreibung welche Endpunkte hat der Server/Backend)), Objektpatterns betreffen Abschnitte im Code (Unterkategorien: Behavioral (Observer, Template Method), structural (Singleton, Composite, Decorator), creational (factory)).

3.5.1 MVC

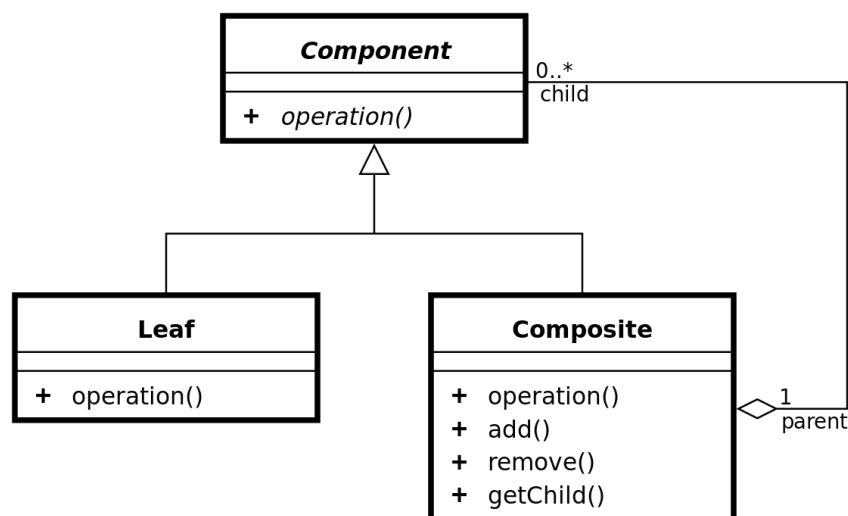
Das Pattern Model-View-Controller ist ein Strukturmuster/Architekturmuster. Es gibt die drei Komponenten: Model, View und Controller. Der Controller verwaltet View und Model. Er ist auch für die Interaktion mit dem Benutzer zuständig. Dabei verändert der Controller die Daten im Modell und erhält per Observer die Änderungen des Benutzers im View. Per Observer erhält der View updates aus dem Model. Der View selber ist ein Kompositum

3.5.2 Observer



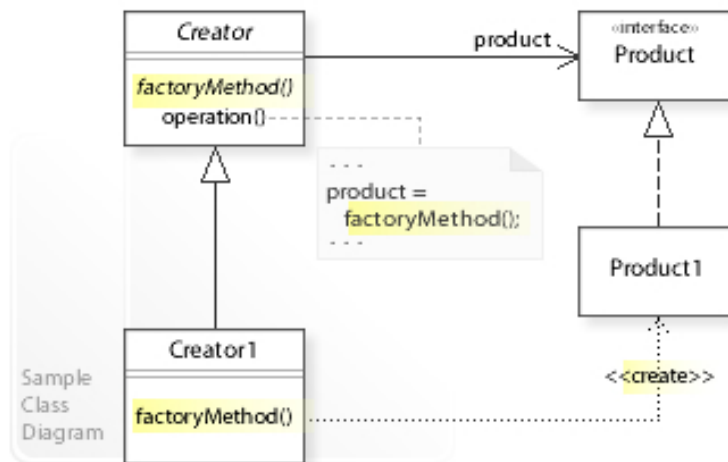
Das Observer-Pattern ist ein Verhaltensmuster. Dabei kann ein Subjekt mehrere Observer halten. Das Subjekt kann neue Observer registrieren und alte abmelden. Dabei ist die Observer-Klasse abstrakt und die speziellen Observer extenden Observer. Sobald sich etwas am Subjekt ändert, können alle Observer notified werden. Diese handeln dann auf ihre eigene Weise.

3.5.3 Composite / Kompositum



Das Kompositum ist ein Strukturpattern und ist eine Liste von Objekten bei der jedes Objekt ein Kind des selben Typs hat und darauf zeigt. Das nennt sich einfach verkettet. Kennt jedes Objekt seinen Vorgänger und Nachfolger, so ist die Liste doppelt verkettet. Zusätzlich kann die Liste einen Start- und/oder einen Endknoten haben, die dann spezielle Varianten des eigentlichen Listenobjekts sind. Die Liste kann zusätzlich auch von einer Oberklasse verwaltet werden.

3.5.4 Factory

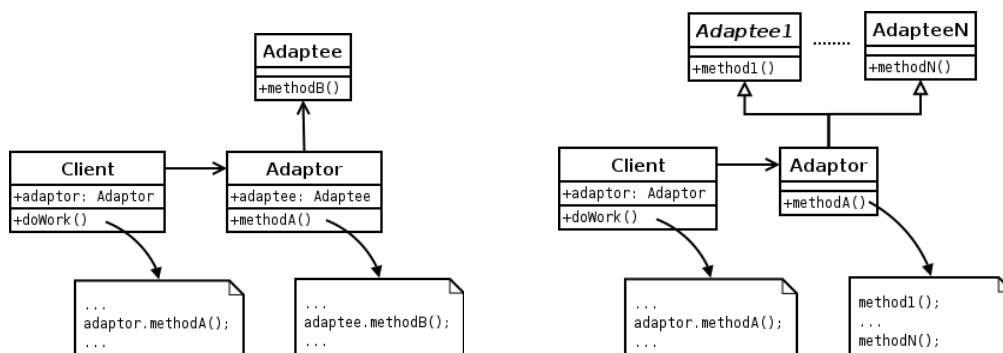


Abstract Factory stellt ein Interface zur Verfügung. Darin steht, was alles erstellt werden kann. (z.B. Stuhl, Sofa, Kaffeetisch). Aber das ist nur die Schnittstelle. Diese hat dann verschiedene Implementierungen: Die Viktorianische Fabrik, die moderne Fabrik und die ArtDeco-Fabrik. Alle drei produzieren jeweils Stuhl, Sofa, Tisch.

Das Factory Pattern ist ein creational pattern. Das bedeutet, dass es um die Erzeugung von Objekten einer Klasse geht. Dabei wird die Frage wie ein bestimmtes Objekt erzeugt wird den jeweiligen extends Klassen der Factory überlassen.

Ein Beispiel: In einem Spiel gibt es Räume (abstrakte Klasse). Das sind entweder normale oder magische Räume. Das bedeutet die beiden Klassen normaler und magischer Raum extenden Raum. Die abstrakte Klasse Game stellt die abstrakte Methode *makeRoom* zur Verfügung. Diese wird dann von den extendenden Klassen MagicGame und NormalGame erst mit Sinn befüllt. Daher wird je nach Struktur eine andere Art von Räumen erzeugt (eben kontextabhängig).

3.5.5 Adapter



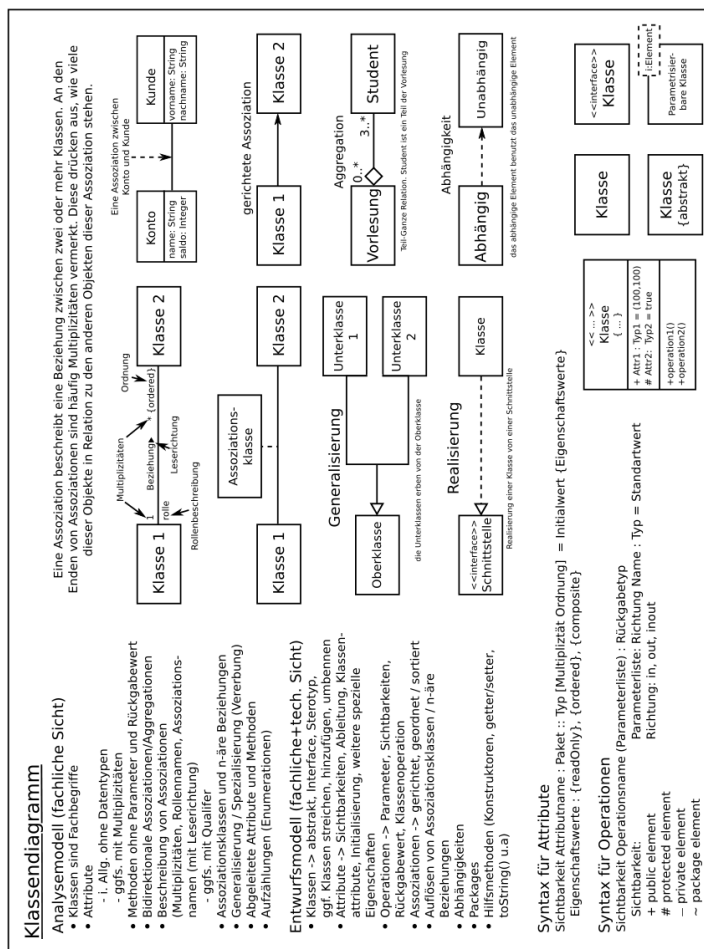
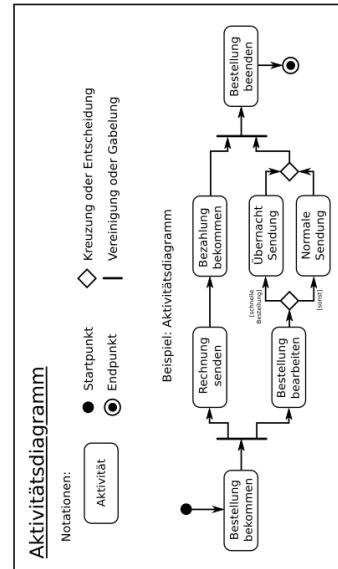
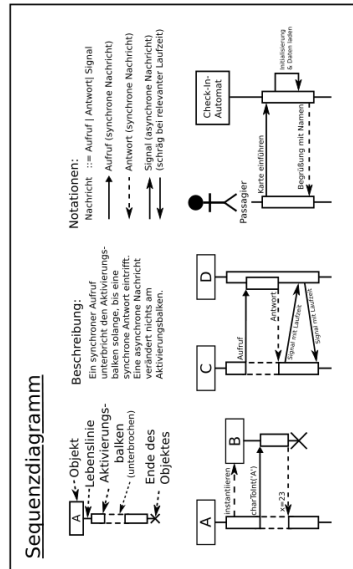
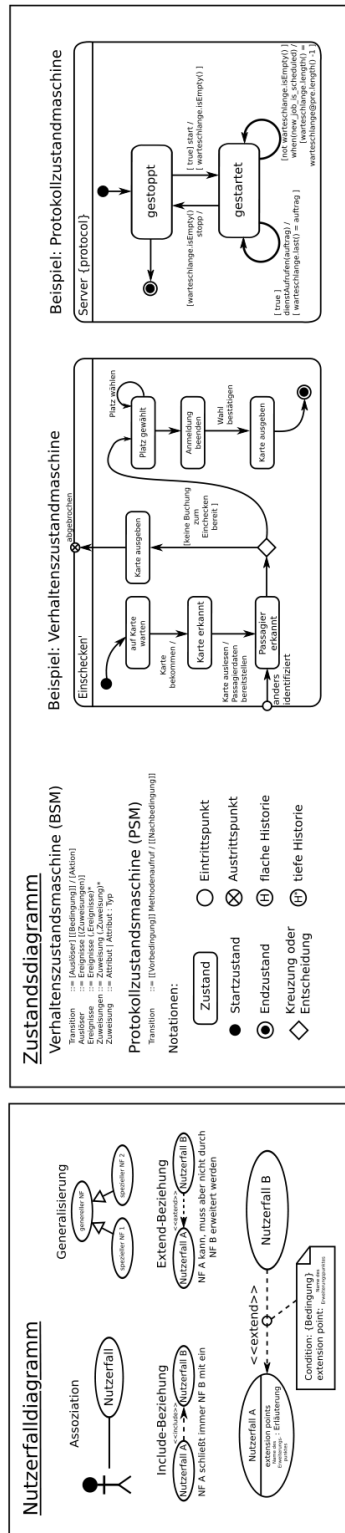
Der Adapter ist ein Strukturalternativ und dient als Schnittstelle/Übersetzer zwischen zwei Klassen/Bibliotheken. Links ist der ObjectAdapter und rechts der ClassAdapter.

3.5.6 Singleton

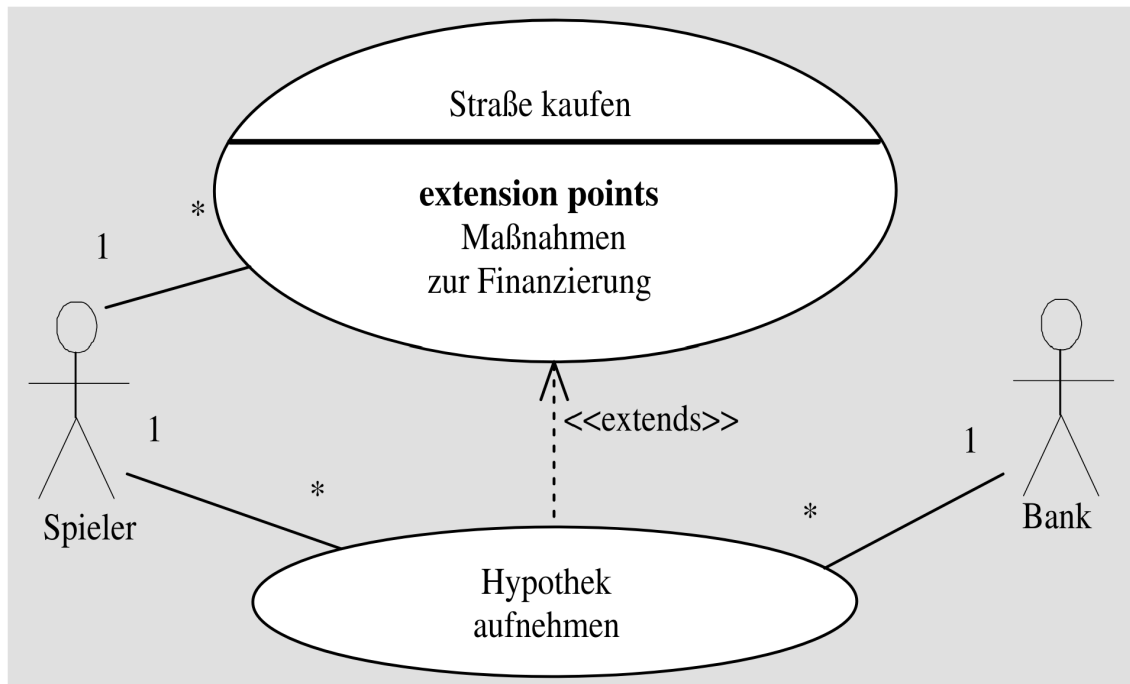
Ein Singleton ist eine Klasse die *final* ist. Es gibt immer nur maximal ein Objekt dieser Klasse. Wird versucht ein neues zu erzeugen, so erhält man das bereits existente zurück.

3.6 UML-Diagramme

Der ultimative UML Überblick:



3.6.1 Use-Case-Diagramme



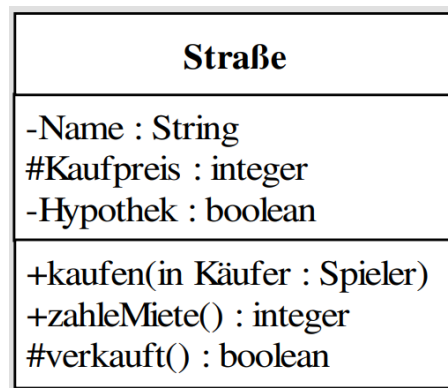
Wichtig für den ersten Schritt im V-Modell (Requirement analysis). Strukturierte Darstellung von Funktionalität (Kein Zusammenhang von Klassen). Wie hängen Funktionen für den Nutzer zusammen. Includes sind garantiert notwendige Bestandteile, Extensions sind Ausnahmen vom Regelfall.

Im Use-Case-Diagramm geht es um die Darstellungen von Handlungen und Aktoren. **Aktoren** können Menschen oder andere Systeme und Geräte sein. **Anwendungsfälle** werden durch Blasen dargestellt. Der Titel sollte direkt beschreiben, was hier passiert.

Jeder Anwendungsfall ist mit mindestens einem Akteur assoziiert. Diese Assoziationen können auch verschiedene Wertigkeiten haben, wie das Beispiel gut zeigt (1 Spieler kann viele Straßen kaufen). Einige Anwendungsfälle müssen genauer beschrieben werden. Das ist durch die folgenden 3 Möglichkeiten realisierbar:

- Mit einem gestrichelten Pfeil und dem Schlüsselwort *«extends»* kann ein Anwendungsfall einen anderen erweitern. Dabei muss die Erweiterung nicht zwangsweise bei jeder Handlung auch verwendet werden. Sie ist optional. Die verschiedenen Erweiterungen heißen **Extension Points** und beschreiben die Erweiterungen. In dem Beispiel gibt es viele Methoden für eine Finanzierung der Straße. Hypothek ist dann eine davon. Es wären aber auch andere möglich.
- Mit einem gestrichelten Pfeil und dem Schlüsselwort *«includes»* kann ein Anwendungsfall einen anderen als grundsätzlich notwendigen Unterfall haben. Ein Beispiel: Bei einem Geldautomaten können verschiedene Aktionen vorgenommen werden. Aber sie alle beinhalten **zwangsweise** die Eingabe der PIN.
- Eine oben beschriebene Erweiterung oder Subroutine kann auch an Bedingungen geknüpft sein. Diese heißen **Conditions**. Sie werden durch Kreisen auf den Erweiterungspfeilen angedeutet. Dabei ist ein Post-it per gestrichelter Linie mit dem Kreis verbunden.

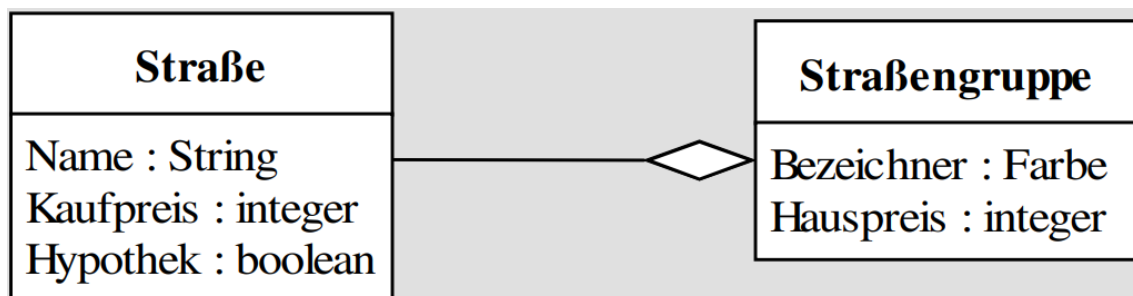
3.6.2 Klassendiagramme



Die Klassenkarte gibt Informationen über die Attribute und Funktionen der Klasse. Sichtbarkeiten werden wie folgt dargestellt: (+) *public*, (-) *private* und (#) *protected*.

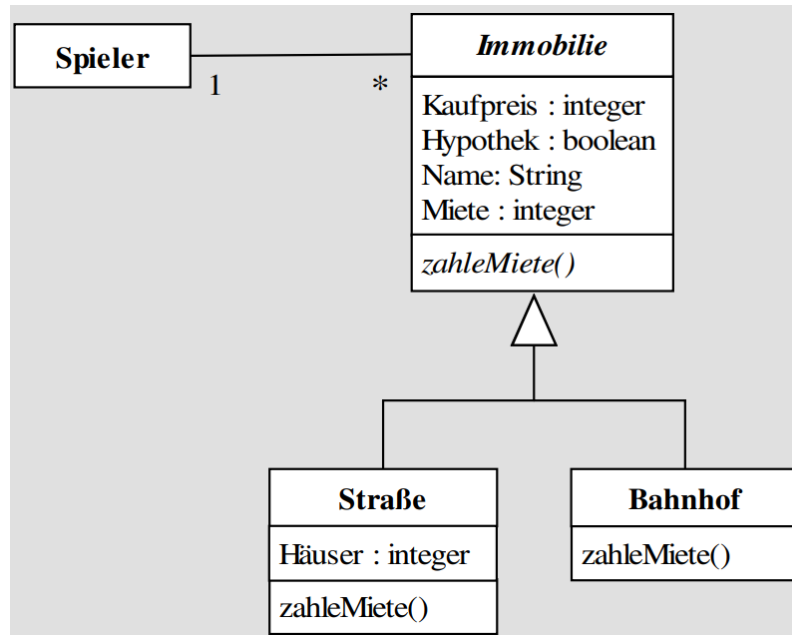
Soll ein Objekt einer Klasse dargestellt werden, so ist der Name unterstrichen. Eine abstrakte Klasse oder ein Interface wird durch kursiv schreiben des Namens angezeigt.

Die Beziehungen werden mit Strichen ausgedrückt. Dabei kann es noch Beschreibungen der Beziehungen geben: Zahlen am Ende einer Linie zeigen an, wie viele Objekte miteinander in Beziehung stehen. Zusätzlich kann Text auf dem Strich die Beziehung weiter beschreiben. Muss eine Beziehung durch eine weitere Klasse beschrieben werden, so wird die Klasse mit gestrichelter Linie an die Relation angehängen. Ist die Abhängigkeit gestrichelt, dann gibt es irgendeinen Bezug, aber man weiß nicht genauer wie es aussieht.



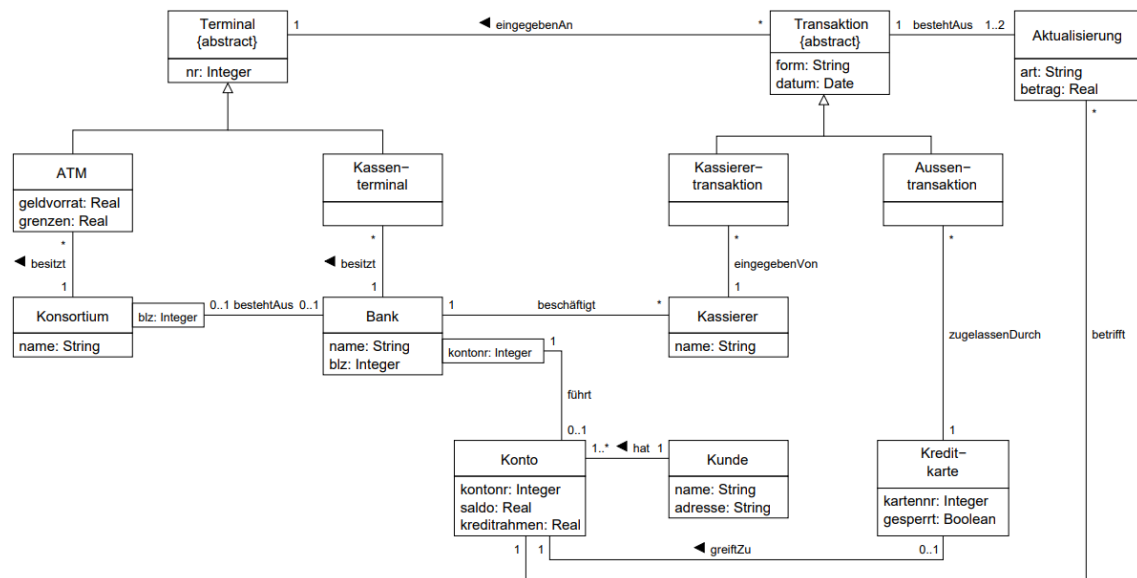
Die Aggregation wird mithilfe einer weißen Raute angezeigt. Sie beschreibt, dass eine Klasse zu ihrem Aggregat gehört ohne von ihm etwas wissen zu müssen. Eine Straße ist eben Teil einer Farbgruppe aber muss über die Gruppe selber nichts wissen.

Eine schwarze Raute ist die Darstellung einer Komposition. Sie ist die strengere Form der Aggregation. Hierbei kann die Unterklasse nicht ohne ihr Aggregat und andersherum existieren. Somit hat dort die Unterklasse den Status eines Attributs, kann aber nach der Erzeugung der Eltern erzeugt und vorher zerstört werden. Also ein Spiel besteht beispielsweise aus 40 Straßen und kann ohne diese nicht sinnvoll existieren.

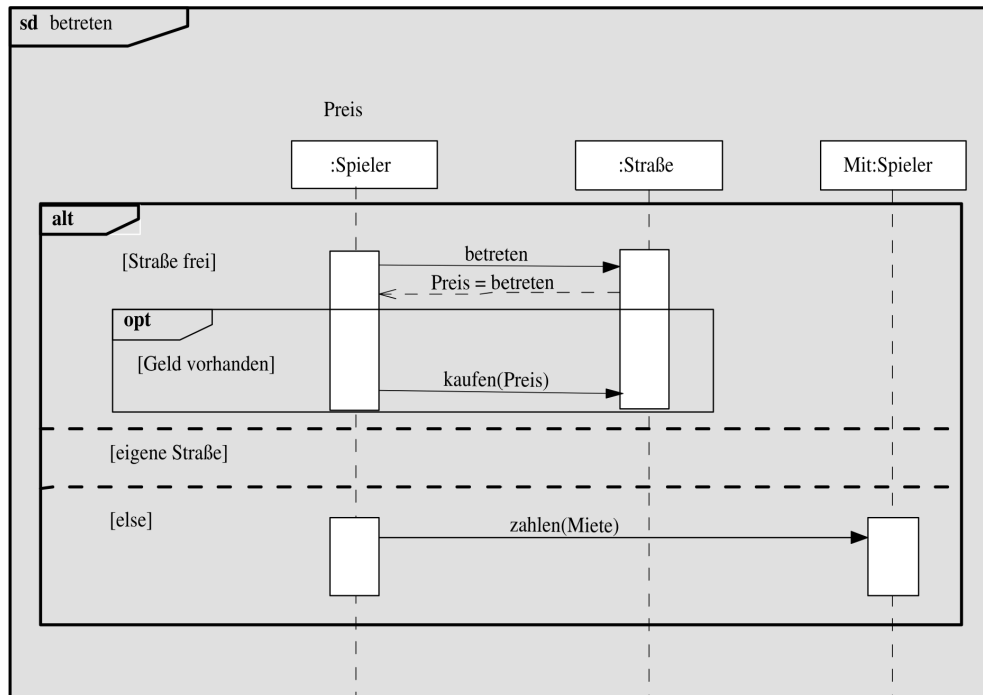


Eine Vererbung wird im Klassendiagramm mit Pfeilen mit weißen Spitzen dargestellt. Handelt es sich um eine abstrakte Klasse als Oberklasse, so sind die Methoden kursiv geschrieben. Handelt es sich um ein Interface, so wird der Pfeil gestrichelt gezeichnet und über dem Klassennamen *«interface»* vermerkt.

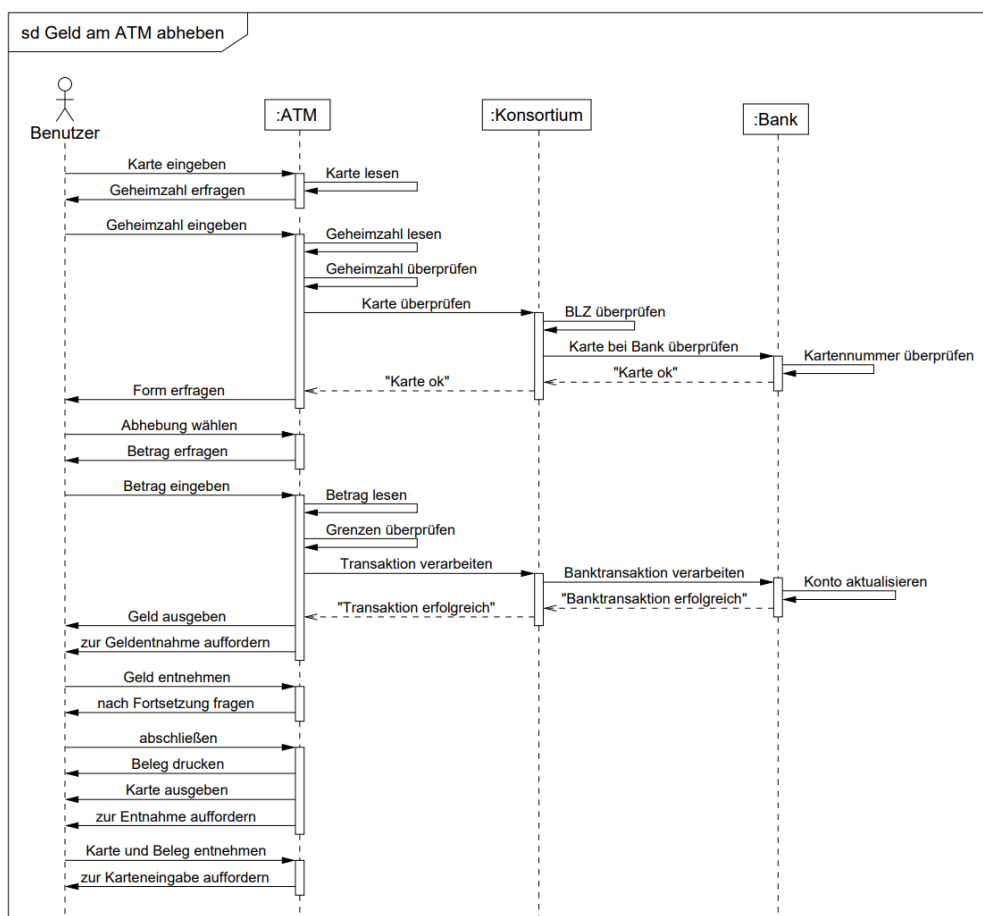
Ein vollständiges Klassendiagramm zeigt das Beispiel aus der Vorlesung:



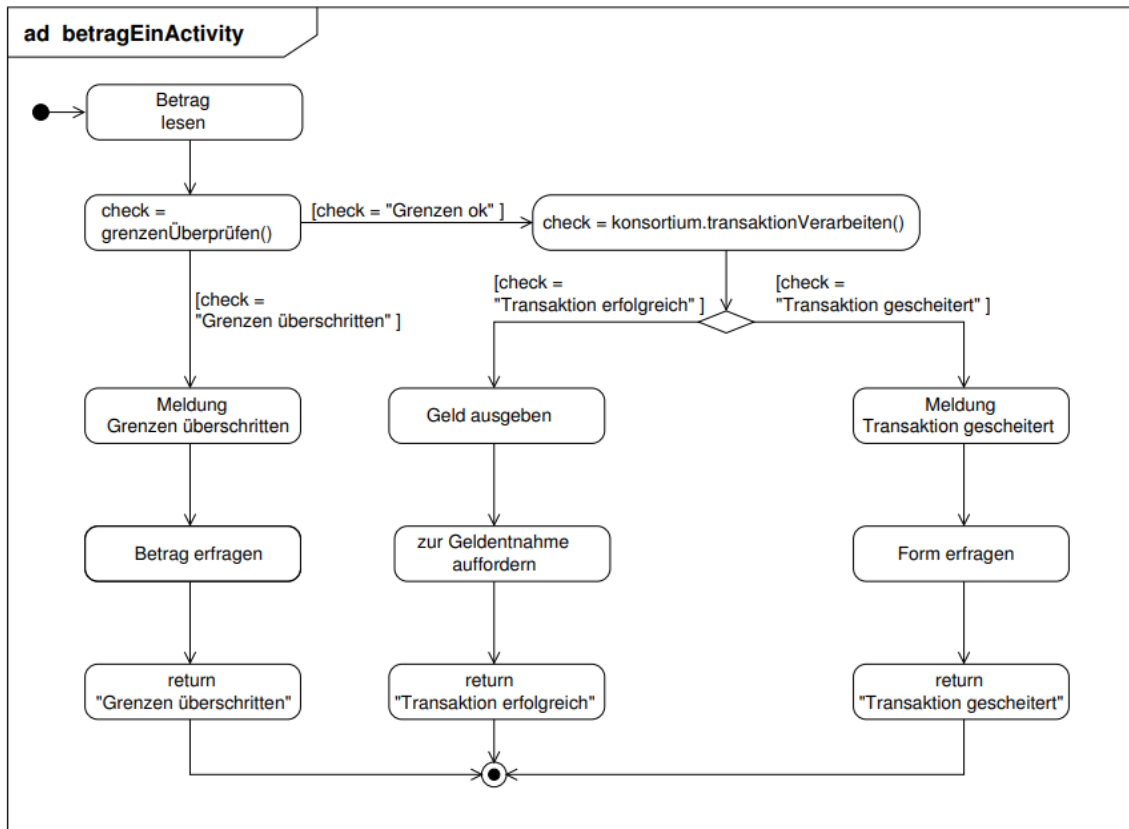
3.6.3 Sequenzdiagramme



Aus der Vorlesung ist das folgende ausführliche Sequenzdiagramm (**Interaktionsdiagramm** in diesem Fall) bekannt:

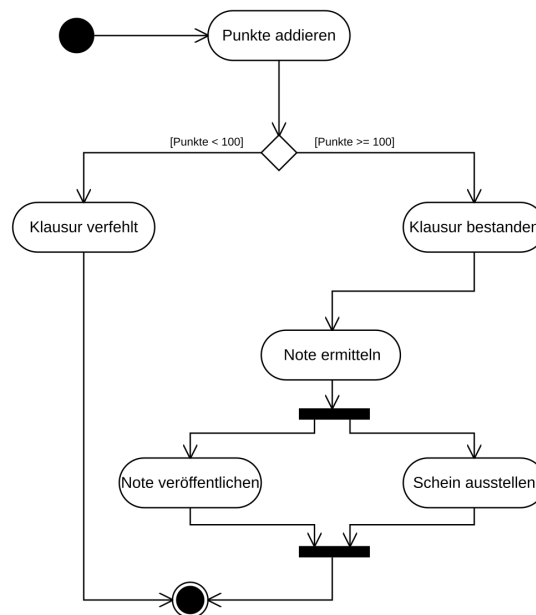


3.6.4 Zustandsdiagramme



Im Zustandsdiagramm werden die Zustände eines Systems modelliert. Dabei ist die Struktur ähnlich den Automaten. Die Übergänge der Zustände werden auf den Pfeilen beschrieben. Optionen ergeben sich durch Rauten. Verfeinerungen einzelner Zustände und Übergänge werden mit *ad* markiert.

3.6.5 Aktivitätendiagramm



3.7 Allgemeine Reste

3.7.1 User Stories

Eine User Story beantwortet die folgende Frage: As **WHO**, do I want **WHAT**, so that **WHY**. Auf der Vorderseite einer User Story steht dann die Beschreibung dieses Problems, also: Wer, was, warum? Auf der Rückseite einer User Story sind die Akzeptanzkriterien gelistet. Diese können dann aber auch deutlich umfangreicher sein, als der User das erwarten würde. Das ist aber dann die Seite für die Entwickler.

Beispiel: Als Student will ich meine Noten einsehen um zu wissen, ob ich die Vorlesung bestanden habe. Auf der Rückseite steht dann: Ein Student kann nur seine Noten einsehen. Die Noten können nicht manuell verändert werden. Und so weiter ...

3.7.2 Refactoring

Beim Refactoring geht es darum, den Code intern zu verbessern, so dass sich an seiner Verhaltensweise nichts ändert. Beziehungsweise nur soweit ändert, dass immer noch die gleichen Anforderungen realisiert bleiben. Zweiteres ist schwer abzugrenzen von einer Neuentwicklung oder einem Neuentwurf.

Ziel ist es, sogenannte Code-Smells los zu werden. Das kann über viele Arten geschehen: Prozeduren abstrahieren, neue Benennungen einführen, Kommentare entfernen/ergänzen, neue Klassen hinzufügen, ungenutzten Code entfernen, zyklomatische Komplexität reduzieren, etc ...

4 Datenbanken

MIN-MAX-Notation, Charakterisierung des schwachen Entitätstypen, Vor-Nachteile eines relationalen Systems, Relationen zusammenfassen/verfeinern, SQL in relationale Algebra kanonisch übersetzen, Dekompositionsalgorithmus, Zyklen in ER-Diagrammen, referentielle Integrität, Datenunabhängigkeit, Trigger, physische Optimierung,

4.1 Das relationale Modell

4.1.1 Begriffe

- Als **Kardinalität** wird die Menge der Einträge einer Relation bezeichnet. Also die Einträge einer BD oder einer Abfrage.
- Die **Stelligkeit** einer Relation ist die Anzahl der verknüpften Domains.
- Das **Schema** bezeichnet die Anordnung und die vorhandenen Attribute (Spalten in einer Abfrage).
- Die **Ausprägung** ist dann die Liste der Elemente, die in der Relation enthalten sind.
- Der **Primärschlüssel** ist die eindeutige Identifikation eines Tupels in einer Relation. Dabei kann ein Schlüssel auch mehrere Attribute umfassen. Hierbei ist darauf zu achten, dass es keine Redundanzen gibt.
- Ein **Fremdschlüssel** ist ein Attribut, das auf den Primärschlüssel einer anderen Relation zeigt.

4.1.2 Vermischte Informationen

- Unter der **Union Compatibility** versteht man das Prinzip, dass zwei Relationen mithilfe des Vereinigungsoperators $\cup$ verbunden werden können. Das bedeutet insbesondere, dass beide Relationen die selbe Anzahl Attribute und vom selben Datentyp haben müssen.
- Die Datentypen **BLOB** und **CLOB** sind Datentypen mit großer Kapazität (Binary/Character Large Object). Sie sind insbesondere dann von Interesse, wenn eine Datenbank viele XML Objekte speichern soll.
- Das 2-Phase Commit protokoll ist eine schwächere Variante des 2PLP (2-Phase Sperrprotokolls). Hierbei erhält ein Koordinator eine Benachrichtigung, dass eine Transaktion durchgeführt werden soll. Er prüft dann für alle beteiligten Instanzen ob diese bereit sind und lässt dann ein commit durch oder bricht mit abort ab.
- **WAL** steht für Write-Ahead-Logging. Hierbei werden Änderungen in situ vorgenommen. Das heißt, dass vor allem Indexstrukturen nicht betroffen sind. Die Protokolle der Änderungen werden dann sequentiell gespeichert, was dazu führt, dass tatsächlich bis zu einem beliebigen Punkt zurück oder vorgesprungen werden kann. Das ist nützlich um für ACID die Consistency hoch zu halten.

4.2 DDL

```
create table Haelt
(
  Dozent      integer      references Dozenten (DNr)
                        on delete cascade,
  VNr         integer      not null,
  Semester    varchar(20) not null,
  primary key (Dozent, VNr, Semester),
  foreign key (VNr, Semester) references Lehrveranst
)
```

Das Schlüsselwort *references* bedeutet, dass auf ein Attribut (*foreign key ... references* den Primärschlüssel) einer anderen Relation gezeigt wird und in dieser nicht ohne weiteres die Elemente entfernt werden können. Gleichzeitig kann aber kein Element in der Relation *Hält* existieren, das kein passendes Paar in der referenzierten Relation besitzt.

Zum Schlüsselwort *cascade* gibt es die Versionen *on update* und *on delete*. Das bedeutet, wenn sich etwas an den Attributausprägungen in der referenzierten Relation ändert oder gelöscht wird, überträgt sich das auch auf die Ausprägung dieser Relation.

4.3 Die Relationale Algebra

4.3.1 Grundoperationen

Die Grundoperationen der relationalen Algebra sind:

Vereinigung	$R = S \cup T$
Differenz	$R = S - T$
Kartesisches Produkt	$R = S \times T$
Selektion	$R = \sigma_F(S)$
Projektion	$R = \pi_{A,B,\dots}(S)$

Dabei ist F ein logisches Prädikat (z.B. $\text{Abteilungsnummer}=01 \wedge \text{Name}=\text{Mayer}$). Die Selektion nach einem kartesischen Produkt ist ein **Join**.

4.3.2 JOINS

Es gibt drei Arten von Joins:

Theta-Join	$R = S \bowtie_{A\Theta B} T$	Θ ist eine Operation (logisch, arithmetisch), A und B sind Attribute.
Equi-Join	$R = S \bowtie_{A=B} T$	Ausprägungen werden verbunden, wenn sie in den Attributen A und B gleich sind. Diese Information fehlt dann in der neuen Relation.
Natural-Join	$R = S \bowtie T$	Die Relationen werden auf allen gleichnamigen Attributen verbunden.

Ein Beispiel dafür ist:

$$R = \pi_{\text{SName}} \left(\text{Städte} \bowtie_{\text{LName}=\text{Land}} \sigma_{\text{Partei}=\text{DCU}}(\text{Länder}) \right) \quad (\text{Relationenkalkül})$$

Es werden alle Städte aus den Ländern ausgegeben, die von der CDU (mit)regiert werden.

4.3.3 Divisionen

Der Divisionsoperator ist eine Spezialität der relationalen Algebra. Er kann in SQL nur aufwändig mit vielen Joins und anderen weiteren Statements reproduziert werden.

Die Aussage $R_1 \div R_2$ bedeutet, dass in der Relation 1 alle Tupel ausgegeben werden, die in den übereinstimmenden Attributen mit der rechten Seite alle Einträge der rechten Relation als Teilmenge haben. Das bedeutet, dass in einer Liste mit Programmierern und ihrer Programmiersprache nur diejenigen Namen ausgegeben werden, die alle Programmiersprachen eingetragen haben, die in der rechten Seite vermerkt sind. (In der damaligen Klausur: Alle Gruppen, die an jedem Hackathon in Hamburg teilgenommen haben)

Eine Möglichkeit der Umsetzung mit korrelierter Unterabfrage:

```
SELECT * FROM suppliers AS s
WHERE NOT EXISTS (
  ( SELECT p.pid FROM parts AS p )
  EXCEPT
  (SELECT sp.pid FROM supplies sp WHERE sp.sid = s.sid )
);
```

Die Query funktioniert wie folgt: Es werden einzeln die Lieferanten durchgegangen. Es werden dann die ausgegeben, bei denen die Subquery leer bleibt. In dieser wird aus der Liste jedes Teil aussortiert, das der aktuell betrachtet Lieferant auch bringt. Ist die Unteranfrage leer, so liefert er alle Teile.

4.3.4 SQL

SELECT

Das SELECT steht am Anfang jeder Suche und entspricht der **Selection** in der relationalen Algebra.

SELECT *	Gib alles aus
SELECT $A_1, A_2, \dots$	Gib nur die Attribute $A_1, A_2, \dots$ aus
SELECT DISTINCT ...	Wie oben nur mit Duplikatelimination
SELECT $A_1 \Theta A_2$ as A_3	verarbeitet die beiden Attribute und benennt sie in der Ausgabe um

FROM

Nach dem Schlüsselwort FROM werden die Relationen angegeben, die miteinander verbunden werden sollen. Dabei ist die Verbindung das Kreuzprodukt. Es können Alias vergeben werden:

```
SELECT *
FROM Mitarbeiter m, Raume r
```

WHERE

Die Einträge nach dem WHERE enthalten logische Operationen, diese können arithmetische Ausdrücke erhalten. So ist es möglich, Werte aus Feldern noch zu manipulieren um sie dann bspw. zu vergleichen.

Eine Besonderheit ist LIKE. Hier wird ein inexakter Vergleich gemacht. Hierbei ist das Prozentzeichen dann ein Platzhalter.

```
SELECT *
FROM Mitarbeiter m, Raeume r
WHERE r.nr LIKE '%4' AND m.name='Jan'
```

Also alle Räume die auf 4 enden im Kreuzprodukt mit allen Mitarbeitern die Jan heißen.

JOIN

Ein JOIN wird dadurch realisiert, dass in der WHERE zwei Attribute miteinander in Gleichheit gebracht werden. Sinnvoll ist das vor allem auf Fremdschlüsseln.

In neueren Dialekten sind auch die folgenden Joins erlaubt und äquivalent:

$A \text{ JOIN } B \text{ ON } A.x = B.x$	
$A \text{ JOIN } B \text{ USING } (x)$	
$A \text{ NATURAL JOIN } B$	Alle gleichnamigen Attribute werden kombiniert

In unserem Beispiel von davor: Nur Informationen zu Räumen und Mitarbeitern werden ausgegeben, die auch von Jan belegt sind.

```
SELECT *
FROM Mitarbeiter m JOIN Raeume r ON r.belegung=m.name
WHERE r.nr LIKE '%4' AND m.name='Jan'
```

JOINS sind nicht immer verlustfrei. Wenn also ein Datensatz aus der linken oder rechten Seite nicht passend gejoint werden kann, fällt er aus dem Ergebnis. Das kann vermieden werden durch LEFT JOIN, RIGHT JOIN und FULL JOIN. Diese sind dann links-/rechtsseitig verlustfrei.

Ein einfaches [INNER] JOIN ist immer auf beiden Seiten mit Verlusten möglich.

Verbindungen

Es können auch mehrere Anfragen (SELECT ...FROM ...WHERE ...) miteinander verbunden werden. Beispielsweise kann damit eine Untermenge von einer Obermenge abgezogen werden.

UNION	Verbindung mit Duplikatelimination
UNION ALL	Verbindung ohne Duplikatelimination
INTERSECT	Schnittmenge
EXCEPT	Subtraktion der zweiten von der ersten Relation
UNION CORRESPONDING	Es werden nur gleiche Attribute übernommen. Dabei kann sich in der zweiten Relation die Reihenfolge ändern

Subquery

Wie später beim Tupel- und Bereichskalkül kann in den bisher gelernten Möglichkeiten ein Quantor nicht explizit verwendet sondern nur im Relationenkalkül simuliert werden. Dafür gibt es die Subquerys. Diese werden zuerst ausgewertet und dann wird damit die Hauptanfrage verfeinert.

Eine Subquery wird in der WHERE Klausel verwendet und wird mit EXISTS aufgerufen. Alternativ zur Simulation eines Allquantors WHERE NOT EXISTS und dann eine negation der Formel im Allquantor. Das Beispiel mit den SPD-Alleinregierungen:

```
SELECT *
FROM Länder L1
WHERE NOT EXISTS (
  SELECT *
  FROM Länder L2
  WHERE L1.LName = L2.LName AND NOT L2.Partei = 'SPD'
)
```

An allen Stellen einer SQL-Query, bei der eine Konstante steht, kann auch eine Subquery eingefügt werden. Bei dieser ist dann **nur ein Tupel und ein Attribut** erlaubt (damit wieder eine Konstante rauskommt). Das nennt man eine **direkte Subquery**.

Es kann auch für einen Vergleich eine Subquery aufgebaut werden. Dafür wird hinter den Vergleichsoperator dann ein ALL, ANY oder SOME geschrieben und dann eine Subquery. In dieser ist dann logischerweise wieder nur ein Attribut erlaubt, hingegen aber mehrere Tupel. Auch damit können dann wieder Quantoren simuliert werden.

Ebenfalls kann ein IN und eine Subquery verwendet werden. Das ist äquivalent zu ANY.

Zueinander äquivalente Querys:

```
SELECT *
FROM MagicNumbers
WHERE NOT EXISTS (
  SELECT Zahl
  FROM Primzahlen
  WHERE Wert=Zahl
)

SELECT *
FROM MagicNumbers
NOT IN (SELECT Zahl FROM Primzahlen)

SELECT *
FROM Primzahlen
WHERE Wert <> ALL (SELECT Zahl FROM Primzahlen)
```

Korrelierte und unkorrelierte Unteranfragen

Eine **korrelierte Unteranfrage** ist eine solche, bei der aus der Hauptanfrage einer oder mehrere Werte berücksichtigt werden müssen. Ein Beispiel ist dabei die Abfrage aus dem Kapitel 4.3.3. Das bedeutet insbesondere, dass diese Abfrage für jedes Ergebnistupel erneut durchgeführt werden muss.

Anders hingegen verhält sich eine **unkorrelierte Unterabfrage**: Diese muss nur ein einziges Mal ausgewertet werden und ist daher deutlich performanter, aber nicht immer geeigneter. Ein Beispiel ist die zweite Abfrage der drei äquivalenten Anfragen aus diesem Unterkapitel. Beide Typen der Unterabfrage kommen meistens in der WHERE Klausel vor, sind aber auch woanders möglich.

Gruppierungen

Sind in einer Relation in einem Attribut gleiche Einträge vorhanden, so können diese gruppiert werden. Die weiteren Projektionen müssen dann Funktionen sein, die die weiteren Attribute gruppiert verarbeiten können (z.B. count(*) oder SUM()). Soll nach diesen neuen Attributen gefiltert werden, kann HAVING verwendet werden. (z.B. HAVING SUM(salary) > 50.000)

Manipulation der Relationen

Mithilfe der Befehle **UPDATE**, **INSERT** und **DELETE** können die Tupel in den Relationen verändert werden.

Das Update kann auf einer Relation durchgeführt werden. Dabei können mehrere Attribute manipuliert werden und dabei andere mit einbezogen werden. Außerdem können die Tupel eingeschränkt werden durch Bedingungen.

```
UPDATE Angestellte
SET Gehalt = Gehalt * 1,02
WHERE Gehalt < 3000
```

Das Entfernen löscht ein ganzes Tupel, das die Bedingungen erfüllt. Sind keine angegeben, so wird die gesamte Relation gelöscht. Dabei kann es zu Kaskaden kommen.

```
DELETE FROM Angestellte
WHERE Gehalt = 0
```

Die neuen Werte werden so eingefügt wie angegeben. Sind dabei nicht alle Attribute benannt, so werden die restlichen Werte mit NULL gefüllt. Ist das nicht möglich, so wird ein Fehler vom DBMS geworfen.

```
INSERT INTO Angestellte (Name,PNr)
VALUES ('Peter',26)
```

4.3.5 Tupel-Kalkül

Beim Tupel-Kalkül gibt es Atome und Formeln. Diese sollen dann zu Ausdrücken zusammengestellt werden. Ganz wie in der Mathematik:

$$\{t \mid \Psi(t)\}$$

Alle Tupel in der Relation, die die Formel Ψ wahr machen. Ein Beispielausdruck kann dann sein:

$$(\forall s \in R)(s.A < u.B \wedge u.C = t.D)$$

Dabei ist s eine gebundene Variable, t bleibt frei und u wird beim zweiten Aufruf gebunden.

Beispiele dafür sind:

$$\begin{aligned}\text{Schema}(t) &= \text{Schema}(\text{Städte}) \ \{t \mid \exists(u \in \text{Länder})(u.LName = t.Land \wedge u.Partei = CDU)\} \\ \text{Schema}(t) &= \text{Schema}(\text{Länder}) \ \{t \mid \forall(u \in \text{Länder})(u.LName = t.LName \implies u.Partei = SPD)\}\end{aligned}$$

Hier sieht man auch die Feinheit der Quantoren: In der ersten Beschreibung werden alle Städte gefunden, die von der CDU (mit-)regiert werden. Die zweite Beschreibung steht für alle Länder, die von der SPD alleine regiert werden.

4.3.6 Bereichskalkül

Im Gegensatz zum Tupelkalkül werden hier nicht die ganzen Tupel angefragt, sondern es kann nur mit den Variablen der Relation gerechnet werden. Dabei sind diese Variable oder auch fest belegbar. Zusätzlich können wieder auch Formeln und logische Prädikate verwendet werden.

$$\{x_1, \dots, x_l \mid \Psi(x_1, \dots, x_n)\} \quad l \leq n$$

Also können damit gezielt Informationen gefiltert werden (wie bei einem SELECT). Die beiden Beispielanfragen wie bereits zuvor in den anderen Kalkülen:

$$\begin{aligned}\{x_1 \mid \exists x_2, x_3, y_1 (\text{Städte}(x_1, x_2, x_3) \wedge \text{Länder}(x_3, y_1, CDU))\} \\ \{x_1 \mid \exists x_2 (\text{Länder}(x_1, x_2, SPD)) \wedge \neg \exists y_3 (\text{Länder}(x_1, x_2, y_3) \wedge y_3 \neq SPD)\}\end{aligned}$$

4.4 Entity-Relationship-Modell

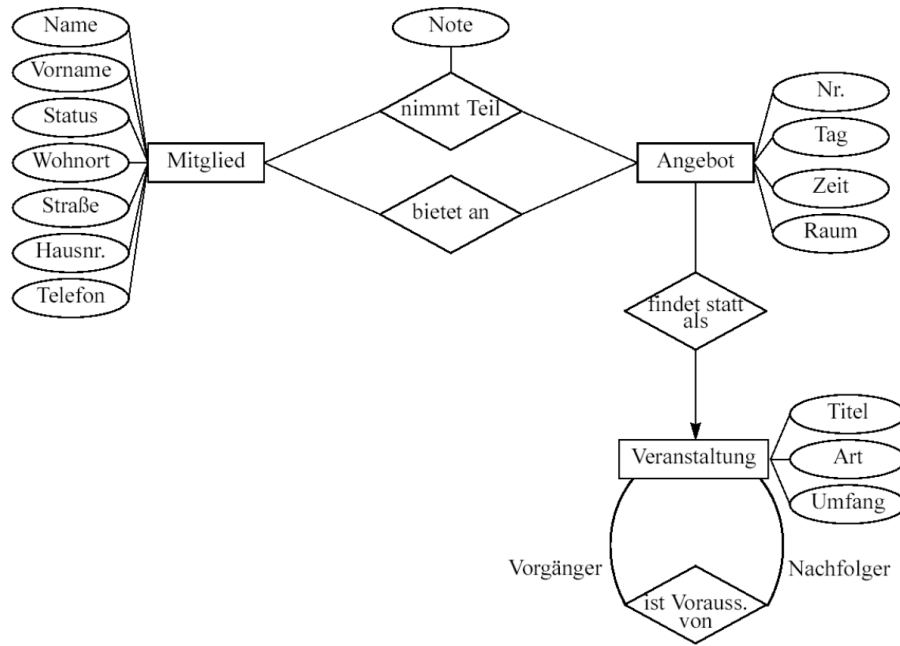
Das Modell ist eine hohe Abstraktion eines Sachverhalts aus der realen Welt. Dabei ist es das Ziel, dieses Modell dann Maschinennah zu transformieren, um es in eine geeignete Datenbank zu überführen. Die gezeigte Notation entspricht der Chen-Notation. Der einzige Unterschied, der noch fehlt, ist, dass es zusammengesetzte Attribute gibt (male Attribute zu Attributen (z.B. Vorname und Nachname zu Name)) und es gibt abgeleitete Attribute. Diese sind in gestrichelten Ovalen gesetzt.

4.4.1 Elemente des E-R-Modells

Es werden drei Elemente unterschieden:

- **Entities:** sind Dinge die auch in der echten Welt existieren. Sie werden hauptsächlich durch Nomen signalisiert. Sie werden in Rechtecken dargestellt.
- **Attribute:** sind Beschreibungen der Entities. Dabei sollen diese durch primitive Datentypen ausdrückbar sein (Int, Char, etc. ...). Primärschlüssel werden unterstrichen. und in runde Blasen gesetzt.
- **Relations:** sind die Beziehungen von Entities zueinander oder zu sich selbst. Die Darstellung erfolgt durch Rauten. Auch Beziehungen können Attribute haben oder zu mehreren Entities eine Beziehung herstellen. Ebenfalls ist eine Beziehung von einer Entity zu sich selber möglich wie im Beispiel: n Veranstaltungen sind Voraussetzung für m andere.

Im folgenden Bild fehlen leider noch die Primärschlüssel.



4.4.2 Funktionalität von Relationen

Um die Beziehung besser auszudrücken gibt es verschiedene Funktionalitäten und deren Darstellung:

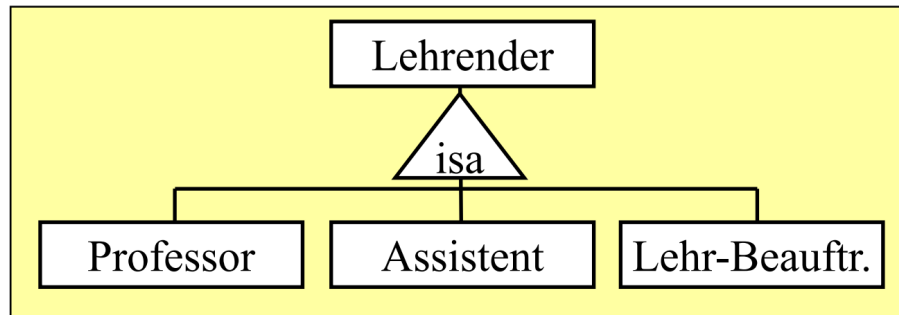
- **1-zu-1 Beziehung:** Hier stehen die Entities in einer genauen Beziehung zueinander. Insbesondere gehört die eine Entity nur zu der anderen. Die Darstellung erfolgt mit einem Pfeil in beide Richtungen oder mit den Zahlen 1 und 1 an beiden Enden.
- **1-zu-n Beziehung:** Hier steht eine Entity zu vielen anderen in Beziehung. Sie werden als Pfeil in Richtung der einzelnen Entity oder den Bezeichnungen 1 und n dargestellt. In dem Beispielbild heißt es also, dass n Angebote als 1 Veranstaltung stattfinden.
- **n-zu-m Beziehung:** Hier stehen beliebig viele Entities auf beiden Seiten zueinander in Beziehung. Hierbei wird einfach nur ein Strich gezogen ohne Pfeilspitze oder n und m an den Enden vermerkt. Im Beispiel: n Mitglieder nehmen an m Angeboten teil.

4.4.3 Schwache Entitäten

Eine schwache Entität ist eine solche, die ohne die Beziehung zu einer anderen Entität gar nicht existieren könnte. Diese werden dann doppelt umrahmt und doppelt verbunden zu der Relation, diese ist ebenfalls doppelt umrahmt. Das Beispiel könnte ein Teil einer Lieferung sein. Ohne die entsprechende Bestellung ist es nicht existent. Es kann aber trotzdem noch weitere Informationen speichern (sowas wie Rabatt). Aber nur das bestellte Produkt ist mit seinem Rabatt völlig wertlos, wenn die Beziehung zur Bestellung fehlt.

4.4.4 Vererbung in E-R

Die Vererbung in dem E-R-Modell wird durch ein Dreieck mit der Bezeichnung *ISA* angezeigt. Das bedeutet, dass alle weiteren Entities die Attribute erben, also die gleiche Ausprägung haben.



4.4.5 Übersetzung E-R zum relationalen Modell

Aus den Entities und ihren Attributen werden Relationen erzeugt. Dabei wird der Primärschlüssel einfach übernommen.

Relationships übersetzen

- Bei einer *one-to-many* Beziehung wird das Primärattribut aus der *one* Seite in die *many* Seite als Fremdschlüssel geschrieben. Die restlichen Attribute bleiben erhalten. Attribute der Beziehung werden ebenfalls in die *many* Relation geschrieben.
- Bei einer *many-to-many* Beziehung wird die Beziehung als neue Relation angelegt. Die Primärschlüssel beider Seiten werden Fremdschlüssel der neuen Relation und bilden zusammen den neuen Primärschlüssel. Weitere Attribute der Beziehung werden als Attribute in die neue Relation eingefügt.
- Bei einer *one-to-one* Beziehung wird wie bei einer *one-to-many* Beziehung gearbeitet. Der Unterschied ist, dass ein Entity komplett aufgelöst wird und in die Relation des anderen geschrieben wird. Dabei wird einer der vorherigen Primärschlüssel der neue Primärschlüssel. Durch die Eindeutigkeit der Schlüssel und der 1:1 Beziehung bleibt auch der neue Primärschlüssel eindeutig.
- Bei mehrstelligen Beziehungen kann verfahren werden wie bei den anderen Beziehungsarten. Bei einem Selbstbezug müssen wieder die Stelligkeit beachtet werden und wie zuvor beschrieben verfahren werden.
- Bei einer *ISA* Beziehung wird für jeden Erben verfahren wie in einer *one-to-many* Beziehung.

4.5 Normalformen

4.5.1 Funktionale Abhängigkeit und Anomalien

Motivation: In einer Relation können viele Doppelungen auftreten. Das heißt, dass beispielsweise einem Lieferanten mehrere Produkte in unterschiedlichen Tupeln zugeordnet wird. Die Redundanzen treten dadurch auf, dass die Stadt und das Land des Lieferanten jedes Mal eingetragen wird. In einem solchen Szenario wird die Abhängigkeit **Funktionale Abhängigkeit** genannt. Da aus der gleichen Stadt auch immer das gleiche Land folgt.

In solchen schlecht entworfenen Relationen können folgende Anomalien auftreten:

- **Update-Anomalie:** Beim Erneuern eines Eintrags (bspw. Adresse) wird nur eines der Tupel verändert. Das heißt, dass alle anderen Tupeln desselben Lieferanten dann falsche Adressen stehen.
- **Insert-Anomalie:** Wird ein neuer Lieferant angelegt, so muss auch immer direkt ein Produkt geliefert werden. Es ist nicht möglich, einfach nur einen Kontakt anzulegen.
- **Delete-Anomalie:** Wird der letzte Eintrag einer Lieferung gelöscht, so wird auch der Lieferant komplett gelöscht. Es ist also nicht möglich, den Lieferant für später abgespeichert zu halten.

Die Idee ist es also, alle funktionalen Abhängigkeiten in eigene Relationen aufzulösen.

Definition: Funktionale Abhängigkeit

Sind X, Y Mengen von Attributen einer Relation R . Dann ist äquivalent:

- Y ist von X funktional abhängig (geschrieben: $X \rightarrow Y$)
- $\forall t, r \in R : t.X = r.X \implies t.Y = r.Y$

Attribute die zu einem Schlüsselkandidaten gehören werden **prim** genannt.

Definition: Volle und partielle Abhängigkeit

Sind X, Y Mengen von Attributen einer Relation R . Dann gilt:

- Y ist von X voll funktional abhängig (geschrieben: $X \twoheadrightarrow Y$), wenn es keine Teilmenge $X' \subsetneq X$ gibt sodass $X' \rightarrow Y$ gilt.
- Andernfalls heißt die Abhängigkeit partiell.

4.5.2 Attributhülle

In der Attributhülle sind alle Attribute gespeichert, von denen es eine funktionale Abhängigkeit zu deren Erzeuger gibt.

Dazu füge alle direkten Abhängigkeiten vom Erzeuger der Hülle hinzu und wiederhole das so lange, bis keine weiteren Abhängigkeiten mehr genutzt werden können.

1. Normalform

Jede relationale Datenbank, jedes relationale Schema ist immer in der 1. Normalform. Dort müssen alle Attribute atomar sein.

2. Normalform

Ein Schema ist in zweiter Normalform, wenn gilt:

- Ein Attribut ist entweder voll funktional abhängig von allen Schlüsselkandidaten oder
- es ist prim.

Somit kann die 2. Normalform nur verletzt sein, wenn ein mehrstelliger Schlüssel(-kandidat) existiert oder wenn es nicht-prime Attribute gibt.

Das Umwandlungsschema von 1. in zweiter Normalform läuft wie folgt: Für jede partielle Abhängigkeit wird die echte Teilmenge X_i des Schlüssels betrachtet, sodass $X_i \rightarrow Y_i$ gilt.

1. Die Tupel mit gleichem X_i werden gruppiert und zusammengeführt.
2. Die Menge Y_i wird entfernt und in eine neue Relation geschrieben.
3. X_i wird der Schlüssel der neuen Relation und verweist auf die alte Relation.

3. Normalform

Ein Schema ist in 3. Normalform, wenn gilt:

- Für jede nicht-triviale Abhängigkeit $X \rightarrow Y$ ist X Teil eines Schlüssel(-kandidaten) oder
- Y ist prim.

Der Umformungsalgorithmus bleibt gleich wie bei der Transition von 1. in 2. Normalform. Die Unterschiede sind:

- Attribute bleiben in der Relation, wenn sie voll funktional abhängig sind vom gesamten Schlüssel und nicht von einem nicht-primen Attribut abhängig sind.
- Es werden auch die Attribute gruppiert und entfernt, die von einem nicht-primen Attribut abhängen. Für diese wird ebenfalls eine neue Relation erzeugt.

Verlustlosigkeit und Abhängigkeitserhaltung

Eine Zerlegung ist **verlustlos**, wenn gilt:

$$R = R_1 \bowtie R_2$$

Eine Zerlegung ist **abhängigkeitserhaltend**, wenn alle funktionalen Abhängigkeiten in mindestens einer der Relationen vollständig enthalten ist, bzw. wenn die Attributhüllen der Abhängigkeiten gleich sind.

$$F_R^+ = \left(F_{R_1}^+ \cup F_{R_2}^+ \right)$$

Kanonische Überdeckung der funktionalen Abhängigkeiten

Hierbei geht es darum aus einer Liste der funktionalen Abhängigkeiten eine neue zu generieren, die möglichst kurz ist.

1. **Linksreduktion:** Prüfe für jede Abhängigkeit ob Teile der linken Seite überflüssig sind. Streiche diese entsprechend.
2. **Rechtsreduktion:** Prüfe, ob eine rechte Seite bereits durch andere (transitive) Abhängigkeiten gegeben ist. Streiche diese rechte Seite.
3. Entferne alle Abhängigkeiten mit leerer rechter Seite.
4. Fasse alle Abhängigkeiten zusammen, deren linke Seite exakt gleich ist.

Synthesalgorithmus für die 3. Normalform

Für eine gegebene Menge F von funktionalen Abhängigkeiten führe die folgenden Schritte aus:

1. Überführe die Menge F in die kanonische Überdeckung F_C .
2. Für jede Abhängigkeit $X \rightarrow A \in F_C$ füge eine neue Relation $R = X \cup A$ hinzu und ordne die verwandten Abhängigkeiten und alle Abhängigkeiten deren Linke und rechte Seiten in der neuen Relation enthalten sind der neuen Relation zu.
3. Rekonstruiere Schlüsselkandidaten: Wenn der Schlüsselkandidat bereits vollständig in einer dieser Relationen genutzt wird, muss nichts getan werden. Andernfalls: Erzeuge eine Relation, die den gesamten alten Schlüsselkandidaten enthält.
4. Eliminiere Relationen, die bereits vollständig in anderen enthalten sind.

Boyce-Codd-Normalform

Definition: Boyce-Codd-Normalform

Eine Relation R ist genau dann in BCNF, wenn für jede nichttriviale Abhängigkeit $X \rightarrow Y$ gilt:
 X enthält einen Schlüsselkandidaten von R .

Was hier gilt: Es gibt keine nichttrivialen Abhängigkeiten zwischen Schlüsselattributen.

Bewertung der Normalisierung

Bei einem klug designten System bedarf es eigentlich keiner oder nur wenig Normalisierung. Das Problem an Normalisierung ist, dass viele Joins ausgewertet werden müssen. Das ist kostenintensiv.

Ein Vorteil ist, dass es die Persistenz der Daten verbessert und den Speicherbedarf verringert.

Mit Integritätsbedingungen (z.B. Triggern) kann man die Anomalien vermeiden. Daher müssen bei der Normalisierung nicht immer alle funktionalen Abhängigkeiten berücksichtigt werden.

4.6 Anfragenoptimierung

Bei der Optimierung von Anfragen geht es darum, dass die Anfrage so umgestaltet wird, dass die Auswertung des Ausdrucks möglichst leicht/schnell/kostengünstig erfolgen kann. Dabei werden drei (im Staatsexamen anscheinend zwei) Arten unterschieden:

- **Logische Optimierung:** Diese Techniken beruhen darauf, die Abfolge der Operationen zu verändern.
- **Physische Optimierung:** Hierbei geht es um die Auswahl der tatsächlichen Algorithmen zur Auswertung/Durchführung einer Operation.
- **Regel- und kostenbasierte Optimierung:** Hierbei wird entweder nach einem festen Regelwerk optimiert oder nach verschiedenen Heuristiken, die es ermöglichen einen kostengünstigen Auswertungsplan zu wählen.

[

Logische Optimierung]Logische Optimierung

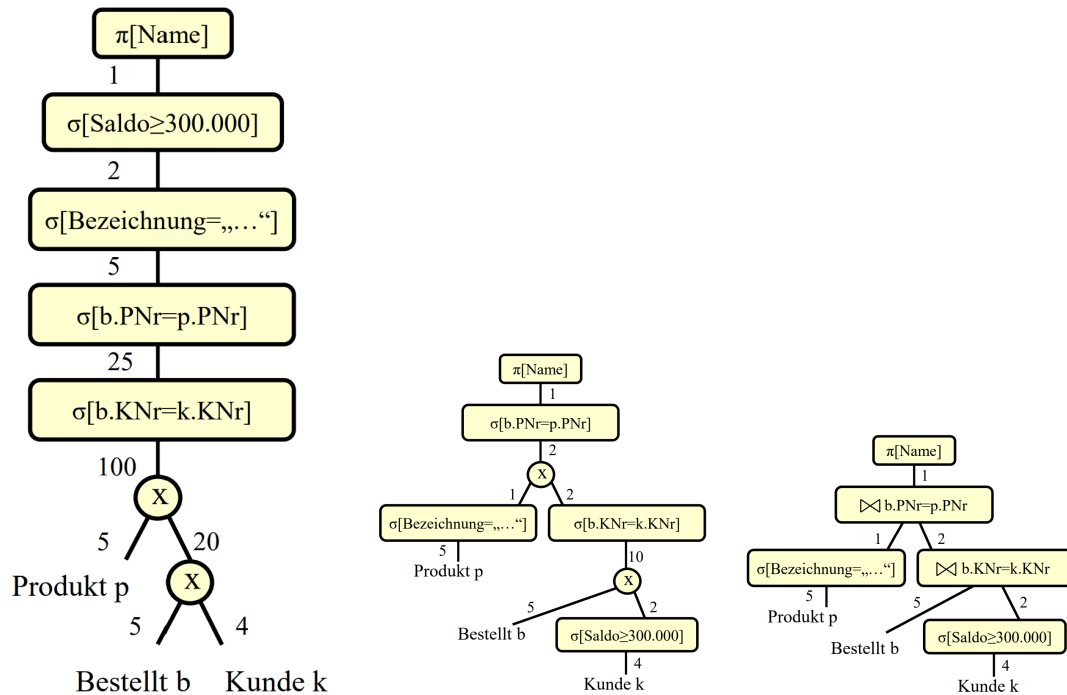
Auf der relationalen Algebra lassen sich einige Äquivalenzregeln betrachten:

- Join, Vereinigung, Schnitt und Kreuzprodukt sind assoziativ und kommutativ.
- Selektionen sind vertauschbar.
- Selektionen mit logischen Operatoren können aufgebrochen werden zu einzelnen Selektionen.
- Projektionen können vernachlässigt werden, wenn sie geschachtelt sind und immer weniger nur noch betrachtet wird nach außen hin.
- Selektion und Projektion sind vertauschbar, solange die Projektion nichts mit der Selektionsbedingung zu tun hat.
- Selektionen nach Produkten können zu Joins gemacht werden.
- Selektionen können mit Joins verschoben werden unter Bedingungen.
- Selektionen können mit den anderen Mengenoperationen verschoben werden, Projektionen nur mit der Vereinigung.

Es gibt ein gutes Rezept zur logischen Optimierung:

1. Aufbrechen der Selektionen
2. Verschieben der Selektionen so weit wie möglich nach unten im Operatorbaum
3. Zusammenfassen von Selektionen und Kreuzprodukten zu Joins
4. Einfügen und Verschieben von Projektionen so weit wie möglich nach unten
5. Zusammenfassen einzelner Selektionen zu komplexen Selektionen

Darstellung der Optimierung im Operatorbaum



4.7 Transaktionen

4.7.1 Eigenschaften

Eine Transaktion folgt immer dem ACID-Prinzip. Das bedeutet:

- **Atomarität:** Eine Transaktion wird entweder ganz oder gar nicht ausgeführt.
- **Consistency:** Eine Transaktion überführt einen konsistenten Datenbankzustand in einen neuen konsistenten Zustand.
- **Isolation:** Eine Transaktion ist blind für die Änderungen anderer Nutzer zur gleichen Zeit. Also wird nur auf einem zu Beginn der Transaktion gültigen Zustand operiert.
- **Dauerhaftigkeit:** Der Effekt einer Transaktion ist dauerhaft (es kann aber durchaus die Möglichkeit zum Rollback geben).

4.7.2 Probleme der Synchronisation von Transaktionen

Da die serielle Ausführung von Transaktionen nicht besonders effizient ist, muss das DBMS eine Möglichkeit der Synchronisation bieten. In diesem Mehrbenutzerbetrieb kann es dann zu den folgenden Anomalien kommen, die das DBMS verhindern muss:

Lost Updates

Zwei Transaktionen überschreiben das gleiche Objekt. Beide haben zuvor den gleichen Zustand gelesen aber nur eines der Updates bleibt erhalten. Das verstößt gegen das Prinzip der **Dauerhaftigkeit**.

Dirty Read und Dirty Write

Diese Anomalie entsteht, wenn eine Transaktion einen Wert ändert und eine weitere Transaktion diesen Zustand liest (dirty Read). Wird dann aber die Transaktion 1 zurückgesetzt und die zweite Transaktion schreibt trotzdem, dann ist das ein dirty Write.

Diese Vorgänge verstoßen gegen die **Dauerhaftigkeit** und die **Consistency**.

Non-repeatable Read

Eine Transaktion liest den Zustand der Datenbank zweimal aus. Währenddessen schreibt eine weitere Transaktion eine Änderung. Da die erste Transaktion noch nicht abgeschlossen ist, wird ein anderer Zustand gelesen.

Das verstößt gegen die **Isolation**.

Phantomproblem

Das Phantomproblem ist eine Variante des Non-repeatable Reads. Dabei fügt die zweite Datenbank ein neues Datum hinzu, während die erste Transaktion noch nicht abgeschlossen ist. Dadurch liest die erste Transaktion dann beim zweiten Read das Phantom mit ein.

4.7.3 Schedules

Es gibt zwei Arten von Schedules: Die **seriellen** und die **allgemeinen** Schedules.

- Ein **serieller** Schedule ist die Hintereinanderausführung von Transaktionen, bei dem die Aktionen nicht verzahnt sind. Diese sind aus Sicht der Isolation und Konsistenz gewünscht.
- Die **allgemeinen** Schedules sind solche, bei denen die Aktionen verschiedener Transaktionen verzahnt ablaufen. Das ist aus Sicht der Performanz gewünscht.

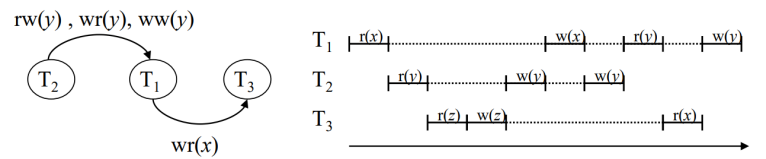
Es muss also der Mittelweg gefunden werden. In diesem Kontext existieren die **serialisierbaren Schedules**.

Beachte dafür folgende Notation:

- $r_i(x)$ bedeutet, dass Transaktion i auf x schreibt.
- $w_i(x)$ bedeutet, dass Transaktion i auf x liest.
- $rw_{i,j}(x)$ bedeutet, dass Transaktion i vor j auf x liest und dann j schreibt.
- $rw_{i,j}(x)$ bedeutet, dass Transaktion i vor j auf x liest und dann j schreibt.
- $ww_{i,j}(x)$ bedeutet, dass Transaktion i vor j auf x schreibt und dann j schreibt.

Ein Serialisierbarkeitsgraph hat alle Transaktionen als Knoten und die Abhängigkeiten (die letzte drei Punkte) werden als Kanten eingetragen. Ist der Graph zyklensfrei, so ist der Schedule serialisierbar.

Beispiel: $S = (r_1(x), r_2(y), r_3(z), w_3(z), w_2(y), w_1(x), w_2(y), r_1(y), r_3(x), w_1(y))$



Ein möglicher Schedule ist dann eine Abfolge von Transaktionen, die aus dem Graphen gelesen werden kann. Dabei kann eine Transaktion nur dann gewählt werden, wenn alle darauf zeigenden Transaktionen bereits durchgeführt wurden.

Um die Serialisierbarkeit zu gewährleisten, ist das 2-Phasen-Sperrprotokoll der Königsweg.

4.7.4 2-Phasen-Sperrprotokoll (2PL)

Das Protokoll verfolgt als Grundprinzip den folgenden Ansatz: Eine Transaktion fordert im Laufe der Zeit immer mehr Sperren an. Werden diese nicht mehr benötigt, so werden die Sperren zurückgegeben.

Möchte eine andere Transaktion auf einen gesperrten Teil (Relation, Attributwert, alles Mögliche denkbar) zugreifen, so muss sie warten.

Dieses Prinzip gewährleistet bereits Serialisierbarkeit. Sie verhindert aber keine Dirty Reads. Das passiert dann, wenn eine Transaktion ihre Sperre zurückgibt und daraufhin eine andere Transaktion auf den entsprechenden Daten liest. Produziert die erste Transaktion später einen Fehler, so muss nicht nur diese, sondern auch alle anderen Transaktionen mit Dirty Reads zurückgesetzt werden.

Daher gibt es das **strikte** 2PL. Hier werden die Sperren erst zurückgegeben, wenn die Transaktion erfolgreich beendet oder zurückgesetzt wurde.

In diesem Kontext fällt auf, dass es bei mehreren Lesern ohne Schreibern nicht zu einer Verletzung der Isolation kommt. Da das strikte 2PL sehr wenig Parallelität erlaubt, werden zwei Typen von Sperren im RX-Sperrverfahren verwendet:

- Lesesperren (R-Locks)
- Schreibsperren (X-Locks)

Das impliziert, dass es möglich ist, auf einem Datum mehrere Lesesperren zu haben, da diese sich nicht gegenseitig behindern.